

Forecasting with Hyper-Trees

Alexander März^{a,*}, Kashif Rasul^b

^a*Independent Researcher*

^b*Morgan Stanley Research*

Abstract

We introduce Hyper-Trees as a novel framework for modeling time series data using gradient boosted trees. Unlike conventional tree-based approaches that forecast time series directly, Hyper-Trees learn the parameters of a target time series model, such as ARIMA or Exponential Smoothing, as functions of features. These parameters are then used by the target model to generate the final forecasts. Our framework combines the effectiveness of decision trees on tabular data with classical forecasting models, thereby inducing a time series inductive bias into tree-based models. To resolve the scaling limitations of boosted trees when estimating a high-dimensional set of target model parameters, we combine decision trees and neural networks within a unified framework. In this hybrid approach, the trees generate informative representations from the input features, which a shallow network then uses as input to learn the parameters of a time series model. With our research, we explore the effectiveness of Hyper-Trees across a range of forecasting tasks and extend tree-based modeling beyond its conventional use in time series analysis.

Keywords: Gradient Boosted Trees, Hyper Models, Hybrid Models, Time Series Forecasting, Time-Varying Parameters

1. Introduction

Gradient boosted decision trees (GBDTs) ([Friedman, 2001](#)) are widely recognized for their accuracy in both regression and classification tasks, while also scaling efficiently to large datasets. Transitioning from their original design as general-purpose models for tabular data, GBDTs have gained prominence in the forecasting domain, demonstrating strong performance across the VN1 ([Vandeput, 2024](#)), M5 ([Makridakis et al., 2022a,b](#)), Corporación Favorita ([Corporación Favorita et al., 2017](#)), and Rossmann ([Knauer & Cukierski, 2015](#)) competitions. In an extensive overview, [Januschowski et al. \(2022\)](#) attribute the success of GBDTs within a time series context to several factors. Key among these are their robustness, relative insensitivity to hyper-parameters, and versatility in handling numerical and categorical features. Further, GBDTs are effective at capturing complex non-linear relationships and interactions, as well as their adaptability to diverse time series characteristics. These attributes position tree-based models as competitive, out-of-the-box forecasting models for effective use across a wide range of time series applications.

Despite their advantages, general-purpose tree-based models are not designed to model sequential data naturally, which can limit their effectiveness in forecasting applications ([Godahewa et al., 2023](#)). Unlike time series models such as ARIMA ([Box et al., 2015](#)) or Exponential Smoothing ([Holt, 1957](#); [Winters, 1960](#)), GBDTs do not inherently capture the sequential structure of time series data. This usually necessitates careful feature engineering to embed time series information effectively into tree-models, e.g., via adding lagged or rolling features derived from the target to be forecasted ([Sprangers et al., 2023](#); [Nasios & Vogklis, 2022](#)). A key reason for this constraint

Preprint submitted to *arXiv*.

*Corresponding author

Email address: alex.maerz@gmx.net (Alexander März)

is that GBDTs operate under the assumption that observations are independent and identically distributed (i.i.d.). Although temporal dependencies can be approximated through engineered features, GBDTs process them in the same manner as any other input, reducing them to just another set of variables used for tree splitting without explicitly recognizing temporal dependencies that characterize time series. As a result, conventional GBDTs cannot naturally incorporate temporal inductive biases, such as autoregressive or trend and seasonality components in Exponential Smoothing models (Holt, 1957; Winters, 1960), without substantial algorithmic modifications. Furthermore, GBDTs face a structural limitation in their forecasting capabilities. Since most trees output piece-wise constants in the leaf nodes, standard implementations of GBDTs are not adept at forecasting beyond the range of the data they have been trained on (Godaheva et al., 2023). A commonly adopted strategy for addressing some of these limitations involves first removing seasonality and trend from the series and then fitting a GBDT to the residuals. The final forecast is then composed of two parts: a local model that forecasts trend and seasonality, e.g., ARIMA (Box et al., 2015), and a tree-based forecast. However, this approach has its own set of challenges. It is less suited for series that lack clear trends or seasonality, becomes complex to maintain for large-scale forecasting applications involving numerous time series, and prevents leveraging cross-series information since separate local models are trained to account for trend and seasonality. To better adapt tree-based models to a time series domain, several approaches have in response been proposed in the literature. Notable contributions include those of Shi et al. (2019) and de Vito (2017), who enhance tree-models by replacing piece-wise constant trees with piece-wise linear trees. This adaptation enables tree-based forecasts to extend beyond the training data range, overcoming the limitations of standard GBDTs. Building on the relationship between threshold autoregressive models and decision trees, Godaheva et al. (2023) introduce a specialized tree algorithm for forecasting. Their approach employs time series-specific splitting and stopping procedures while training global pooled regression models in the leaves to leverage cross-series information.

These advancements have markedly improved the ability of GBDTs to better adapt to a time series context. Nevertheless, effective modeling of temporal dynamics using GBDTs presents ongoing challenges and remains an active area of research. To further progress in this domain, we propose a novel approach that extends tree-based models more naturally to a time series context via the use of Hyper-Trees. Unlike conventional tree-based approaches that forecast time series directly, Hyper-Trees generate forecasts through the following jointly trained process: the tree-component learns the parameters of a classical time series model as functions of input features, which the target model then uses to generate the final forecast. By reformulating GBDTs into a Hyper-Tree architecture, we combine the effectiveness of GBDTs on tabular data with classical forecasting models, such as ARIMA (Box et al., 2015) or Exponential Smoothing (Holt, 1957; Winters, 1960). While our framework is built upon the well-established LightGBM model (Ke et al., 2017), it can in principle be used with any modern GBDT framework. To illustrate how our Hyper-Tree framework differs from and extends conventional time series forecasting, we contrast it with a traditional $AR(p)$ model in the following using the **Rossmann Store Sales** dataset, which we analyze in more detail in Section 4. Rossmann operates over 3,000 drug stores across 7 European countries, and the task of the competition was to forecast 6 weeks of daily sales for 1,115 stores located in Germany (Knauer & Cukierski, 2015). The data is characteristic of the complexity typical in operational time series forecasting (Januschowski & Kolassa, 2019), with a rich set of features beyond basic

time series information, including store types, sales campaigns, holiday information, as well as various categorical features. Conventional applications of an $\text{AR}(p)$ model would require training a separate, local model for each store $i = 1, \dots, N$

$$\begin{aligned} \text{Store 1 : } y_{1t} &= \theta_{1,1}y_{1t-1} + \theta_{2,1}y_{1t-2} + \dots + \theta_{p,1}y_{1t-p} \\ &\vdots \\ \text{Store N : } y_{Nt} &= \theta_{1,N}y_{Nt-1} + \theta_{2,N}y_{Nt-2} + \dots + \theta_{p,N}y_{Nt-p} \end{aligned} \quad (1)$$

with each store being forecasted in isolation and no information sharing across stores or series. In contrast, our proposed Hyper-Tree framework combines the strengths of GBDTs for modeling tabular data (categorical features, missing values, feature interactions and non-linear relationships, ...) with the effectiveness and simplicity of traditional time series models. Specifically, we assume an autoregressive structure for this example, with the Hyper-Tree serving as a global model that learns the $\text{AR}(p)$ parameters as functions of temporal and store-specific features via gradient boosted trees

$$y_{it} = \theta_{1,it}(\mathbf{x}_{it})y_{it-1} + \theta_{2,it}(\mathbf{x}_{it})y_{it-2} + \dots + \theta_{p,it}(\mathbf{x}_{it})y_{it-p}, \quad i = 1, \dots, N \quad (2)$$

where $\mathbf{x}_{it} = \{\text{store-type}_i, \text{region}_i, \text{sales-campaign}_{it}, \text{holiday}_{it}, \text{week}_t, \text{month}_t, \dots\}$. Among recent efforts to adapt gradient boosted trees for time series tasks, Hyper-Trees offer a new perspective by combining classical statistical models with machine learning. Rather than using GBDTs to forecast directly, we repurpose them to learn the parameters of a target time series model. This hybrid formulation offers several advantages for time series analysis and forecasting:

- **Improved Extrapolation in Tree-Based Models** Hyper-Trees can forecast beyond the value range observed during training, overcoming a key limitation of standard GBDTs. Conventional GBDTs output piece-wise constant values, which limits their extrapolation capability compared to parametric models that can naturally extend beyond the training data range (Godahewa et al., 2023). This limitation is particularly problematic in scenarios where sales need to be projected beyond historical ranges. For instance, a store launching a new sales campaign might expect sales significantly higher than any historical values, or a newly renovated store might show stronger growth trends than observed historically. While Light-GBM (Ke et al., 2017) allows fitting linear (Shi et al., 2019) rather than piece-wise constant trees to partially address this issue, other GBDT implementations lack this functionality. To overcome these limitations, Hyper-Trees generate forecasts through the following jointly trained process: first, they model the $\text{AR}(p)$ parameters $\theta_{j,it}(\mathbf{x}_{it})$ for $j = 1, \dots, p$ and each store $i = 1, \dots, N$ based on its specific characteristics $\mathbf{x}_{it}$ (e.g., store type, location, sales campaign, ...). Then, these parameters are applied to the target model to generate the final forecast. This approach allows Hyper-Trees to leverage the inductive bias of the target model, enabling forecasts to naturally extend beyond historical ranges.
- **Cross-Series Learning with Local Adaptivity** Hyper-Trees offer a framework that incorporates tree-based cross-series learning while preserving local adaptivity of classical models. Traditional approaches like ARIMA (Box et al., 2015) or Exponential Smoothing (Holt,

1957; Winters, 1960) typically require individual model training for each time series. Hyper-Trees, in contrast, combine global tree-learning with local model adaptivity. Rather than estimating separate parameters for each store in isolation, Hyper-Trees learn a global mapping from features to target model parameters $f : \mathcal{X}^C \rightarrow \Theta^P$, where $\mathcal{X}^C$ represents the C -dimensional feature space of temporal and store-specific characteristics and Θ^P represents the P -dimensional parameter space of the target model. This enables stores with similar characteristics to share information, which is particularly valuable when forecasts for a new store with limited history can leverage temporal dynamics learned from existing locations. Despite this global approach, Hyper-Trees preserve local adaptivity through store-specific features. While all stores are modeled jointly, each receives unique parameters based on its individual characteristics. For instance, a store in a high-competition area might have parameters that emphasize sales campaign effects, while a store near schools might have parameters that reflect holiday-related patterns.

- **Time-Varying Parameters** Hyper-Trees enable dynamic parameter estimation that adapts to changing conditions. Unlike classical time series models that estimate static parameters, Hyper-Trees generate time-varying coefficients $\theta_{j,it}(\mathbf{x}_{it})$ at each time point. These parameters vary both cross-sectionally through store-specific features like store type and region, as well as temporally through features such as week, month, and year. As an example, Hyper-Trees can learn distinct $\text{AR}(p)$ coefficients for weekday versus weekend patterns, automatically adjusting to capture different sales dynamics throughout the week. By updating its coefficients at each time step as new information becomes available, Hyper-Trees provide a flexible framework that responds to evolving environments (Lee et al., 2023).
- **Parameter Estimation** Hyper-Trees provide a more comprehensive approach to parameter estimation. While conventional estimation of ARIMA (Box et al., 2015) or Exponential Smoothing (Holt, 1957; Winters, 1960) models typically relies on first-order optimization methods using gradients only, Hyper-Trees leverage both gradient and Hessian information, potentially improving on parameter estimates. Furthermore, unlike estimation of classical linear time series models, Hyper-Trees learn parameters as non-linear functions of features, allowing for more flexible parameter estimation in complex, high-dimensional feature settings.
- **Model Transparency and Interpretability** Hyper-Trees provide a transparent modeling framework by combining the flexibility of gradient boosted trees with the interpretability of classical time series models. While the GBDT component learns parameters by modeling complex feature interactions, final forecasts are generated through well-established time series models whose parameters retain clear statistical and domain-specific meaning. Furthermore, practitioners can trace how these parameters evolve in response to changing conditions or external events. This level of transparency not only facilitates understanding of how the model arrives at its outputs but also aligns with interpretability requirements in high-stakes domains such as insurance, healthcare, and public policy (Rudin, 2019).
- **Full Functionality of GBDTs** Hyper-Trees maintain compatibility with established GBDT frameworks like LightGBM (Ke et al., 2017), preserving their full functionality within our

framework. This includes handling missing values, feature importance analysis¹, as well as efficient processing of categorical and numeric features. Additionally, our framework supports imposing structural assumptions on estimated parameters, enabling monotonic time series patterns to be modeled either through GBDTs’ built-in monotonicity constraints or by applying restrictions directly on parameter estimation. This compatibility enables practitioners to combine existing GBDT capabilities with classical time series modeling.

To maintain the scalability of our approach when estimating a large set of target model parameters, we propose a hybrid architecture that combines GBDTs and neural networks within a single framework. This approach preserves the strong feature learning capabilities of GBDTs for tabular data while leveraging the efficiency of neural networks in multi-output tasks. Traditionally, GBDTs use a one-vs-all strategy, estimating one tree per parameter, which can be computationally expensive with an increasing number of target model parameters (Iosipoi & Vakhrushev, 2022). In contrast, our hybrid approach employs GBDTs to first generate informative representations from the input features, which are then decoded by a shallow neural network into the parameters of a target time series model. Both the GBDTs and the neural network are trained jointly, enabling end-to-end optimization and the integration of tree-based feature learning with neural-network-based parameter mapping. This contrasts with most existing approaches where GBDTs are trained first and then a neural network is trained separately.

The remainder of this paper is organized as follows: Section 2 introduces the Hyper-Tree framework and describes its integration with different target model classes. Section 3 gives an overview of related works, while Section 4 presents the application of Hyper-Trees across various datasets. Section 5 provides a conceptual analysis of our Hyper-Tree framework. Finally, Section 6 concludes. Our exploration of Hyper-Trees aims not only to lay a foundation for future research but also to broaden the perspective on the potential of tree-based models beyond their conventional use in time series analysis and forecasting.

2. Hyper-Trees

Hyper-networks, introduced by Ha et al. (2017), have proven to be a valuable concept in machine learning. These networks are designed to generate parameters for a target network, with their key feature being that the generated parameters are modeled as functions of features. The integration of hyper-networks has represented a considerable advancement in the field, offering new possibilities for adaptive and context-dependent modeling. Such integration enables the target network to adapt more seamlessly to changes in data patterns, without the need to retrain the target model (Lee et al., 2023). While hyper-models have shown notable potential, they have primarily been explored within the realm of deep learning. In this paper, we recast this concept for GBDTs in time series forecasting. We term our framework Hyper-Trees, drawing a parallel to hyper-networks. Just as hyper-networks generate parameters for a target network, Hyper-Trees learn to generate parameters for a target time series model. The gradient-based nature of GBDTs allows the target model to

¹Though SHAP values (Lundberg et al., 2020; Lundberg & Lee, 2017) are supported to investigate feature effects on parameter estimates, this functionality is currently restricted by the LightGBM implementation and is only available when the linear tree option (Shi et al., 2019) is not used (as of LightGBM version 4.6.0).

be a member of a wide class of time series models, e.g., ARIMA (Box et al., 2015), Exponential Smoothing (Holt, 1957; Winters, 1960), as well as other target model architectures. By training the Hyper-Tree and applying the estimated parameters to the target model, our approach flexibly and efficiently incorporates time series inductive biases into tree-based models. Compared to hyper-networks, which often require large amounts of data and are prone to overfitting, modeling the parameters of a target model via GBDTs offers a more efficient and robust approach, particularly in settings with limited data, as commonly encountered in real-world settings (Bansal et al., 2022; Vogelsgesang et al., 2018). While applicable to a wide range of forecasting tasks, our Hyper-Tree framework is also well-aligned with operational time series data and forecasting (Januschowski & Kolassa, 2019), which typically involves high-dimensional feature sets that extend beyond basic time series information, including a mix of continuous and categorical features.

2.1. Gradient Boosted Decision Trees

While gradient-based parameter estimation is well-established for neural networks, using GBDTs to learn the parameters of a target time series model is less common and perhaps non-obvious. To better understand how GBDTs can be repurposed for learning parameters of target time series models, we briefly review their gradient-based nature. Modern GBDT implementations such as XGBoost (Chen & Guestrin, 2016), LightGBM (Ke et al., 2017), and CatBoost (Prokhorenkova et al., 2018) rely on gradients g_i and Hessians h_i , where

$$g_i = \frac{\partial \mathcal{L}(y_i, \hat{\psi}_{i_{\mathcal{T}}})}{\partial (\hat{\psi}_{i_{\mathcal{T}})}}, \quad h_i = \frac{\partial^2 \mathcal{L}(y_i, \hat{\psi}_{i_{\mathcal{T}}})}{\partial (\hat{\psi}_{i_{\mathcal{T}}})^2} \quad (3)$$

are the first and second order derivatives of a loss function $\mathcal{L}$ with respect to the output $\hat{\psi}_{i_{\mathcal{T}}}$ of tree $\mathcal{T}$, for observations $i = 1, \dots, N$. Among others, gradients and Hessians serve two crucial functions in each boosting iteration: they guide the tree construction via split decisions and determine the optimal values assigned to leaf nodes via

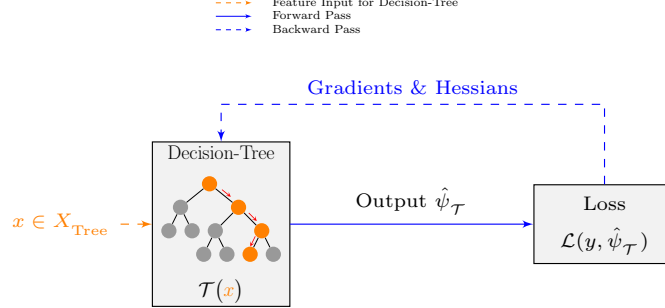
$$w_j^* = -\frac{G_j}{H_j + \lambda}, \quad \text{with} \quad G_j = \sum_{i \in I_j} g_i, \quad H_j = \sum_{i \in I_j} h_i \quad (4)$$

where $I_j = \{i | q(x_i) = j\}$ is the set of indices of observations assigned to the j -th leaf, $q(\cdot)$ being the learned tree structure that maps an input x to its corresponding leaf j , and λ is a regularization term (Chen & Guestrin, 2016). For split decisions, a greedy approach is used that maximizes the loss reduction

$$\mathcal{L}_{split} \propto \frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \quad (5)$$

where I_L and I_R denote the instance sets of left and right nodes after a candidate split respectively and $I = I_L \cup I_R$ represents their union (Chen & Guestrin, 2016). The use of gradient and Hessian information, as illustrated in Figure 1, provides GBDTs with considerable flexibility and enables their reformulation for parameter learning, which we outline in more detail below.

Figure 1: Conventional GBDT architecture showing feature input, decision tree processing, output generation, and loss calculation, with backward pass for gradient-based optimization.



2.2. Hyper-Tree Architecture

Conventional GBDTs operate in function space, mapping input features x to outputs $\hat{\psi}_\tau$ by minimizing a specified loss function. For instance, when using Mean Squared Error (MSE) in a regression setting, $\hat{\psi}_\tau$ represents the conditional mean of the target variable. While the use of an $\mathcal{L}_2$ -type loss may suggest that GBDTs create outputs by directly averaging target values, they instead operate through gradient and Hessian-based updates of the following form²

$$\hat{\psi}_{i_\tau}^{(m)}(x_i) = \hat{\psi}_{i_\tau}^{(m-1)}(x_i) + \hat{\delta}_{i_\tau}^{(m)}(x_i), \quad \text{where} \quad \hat{\delta}_{i_\tau}^{(m)}(x_i) = \nu \cdot w_{j(i)}^* \quad (6)$$

where $\hat{\psi}_{i_\tau}^{(m)}$ denotes the output after $m = 1, \dots, M$ iterations, $\hat{\delta}_{i_\tau}^{(m)}$ is the incremental update at iteration m , ν is the learning rate, and $w_{j(i)}^*$ is the weight assigned to the leaf $j(i)$ corresponding to observation i , where the leaf assignment is determined by the feature vector x_i through the structure of the learned tree, i.e., $j(i) = q(x_i)$. The leaf weights approximate a Newton-Raphson update (Sigrist, 2021), where $w_j^* = -\frac{G_j}{H_j + \lambda}$ aggregates individual gradients and Hessians across all observations in leaf j . This update step makes GBDTs highly adaptive for a wide range of tasks beyond conventional forecasting, allowing $\hat{\psi}_\tau$ to represent any quantity as long as the associated loss function is twice continuously differentiable.

By conceptualizing GBDTs as Hyper-Trees, we shift from performing gradient descent in function space, where the tree output $\hat{\psi}_\tau$ is a forecasted value, to gradient descent in parameter space, where the tree output represents parameters of a target time series model, $\hat{\psi}_\tau \equiv \hat{\theta}_\tau$. In our framework, Hyper-Trees are trained by solving

$$\min_{\hat{\theta}_\tau} \mathcal{L}(y, \mathcal{M}_{\hat{\theta}_\tau}(y)) \quad (7)$$

where the loss $\mathcal{L}$ is evaluated on the forecast $\mathcal{M}_{\hat{\theta}_\tau}(y) = \hat{\psi}_\mathcal{M}$ of the target time series model $\mathcal{M}_{\hat{\theta}_\tau}$. Since $\hat{\psi}_\mathcal{M}$ is a composition of the target model and the tree-generated parameters, gradients and Hessians are obtained by applying the chain rule through $\mathcal{M}_{\hat{\theta}_\tau}$. This requires the target model

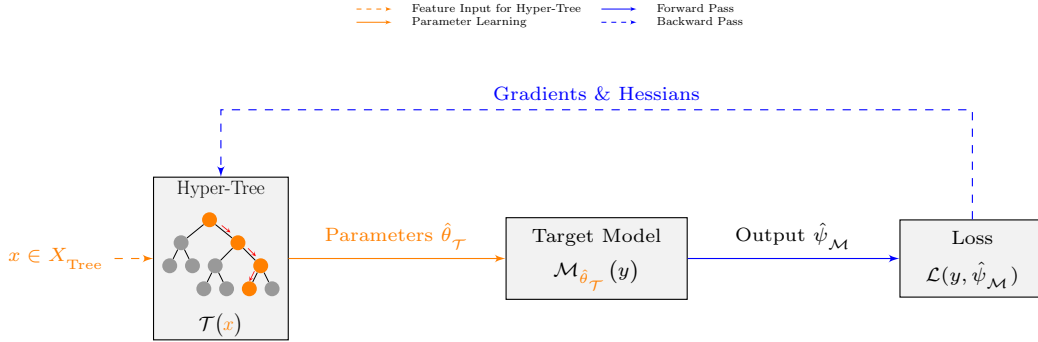
²For the specific case of MSE loss, the gradient is proportional to the residuals, and the Hessian is constant. This can lead to the impression that GBDTs simply fit residuals, when in fact they always follow gradient-based updates derived from the specified loss function.

$\mathcal{M}_{\hat{\theta}_{\mathcal{T}}}$ to be twice continuously differentiable with respect to its parameters, as the GBDT update relies on both first- and second-order derivatives. For the j -th parameter estimated by the Hyper-Tree and observations $i = 1, \dots, N$, the gradient and Hessian are given by

$$g_i^{(j)} = \frac{\partial \mathcal{L}(y_i, \mathcal{M}_{\hat{\theta}_{i\mathcal{T}}}(y_i))}{\partial (\hat{\theta}_{i\mathcal{T}}^{(j)})}, \quad h_i^{(j)} = \frac{\partial^2 \mathcal{L}(y_i, \mathcal{M}_{\hat{\theta}_{i\mathcal{T}}}(y_i))}{\partial (\hat{\theta}_{i\mathcal{T}}^{(j)})^2} \quad (8)$$

Consistent with the standard multi-output GBDT strategy, a separate tree is grown for each parameter, and only the diagonal entries of the Hessian matrix are required. Due to its modularity, where both the Hyper-Tree and the target model can be modified as required by the specific characteristics of the data, gradients and Hessians typically do not have an analytical solution. We therefore leverage the automatic differentiation capabilities as available in PyTorch (Paszke et al., 2019) for deriving gradients and Hessians. The architectural design of our approach is illustrated in Figure 2 with the Hyper-Tree as the central component.

Figure 2: Hyper-Tree architecture illustrating a unified framework where a Hyper-Tree generates parameters for a target model. The output of the target model is passed to a loss function, with gradients and Hessians flowing back, enabling learning of temporal dependencies and integration of diverse feature types.



The Hyper-Tree $\mathcal{T}$ takes a set of input features x and generates parameters $\hat{\theta}_{\mathcal{T}}(x)$ that are used by a target model $\mathcal{M}$ to generate a forecast $\hat{\psi}_{\mathcal{M}}$ which is passed to a loss function $\mathcal{L}$. Gradients and Hessians, derived from the loss function with respect to the target model parameters, guide the Hyper-Tree to learn parameters optimized to improve the accuracy of the target time series model. This architecture enables integrated training and ensures that the loss incorporates information from both components: the parameters generated by the Hyper-Tree and the forecasts created by the target model. For the architecture shown in Figure 2, the temporal component can, for example, be modeled via an $\text{AR}(p)$ process, though the target model $\mathcal{M}$ may belong to a broad class of time series models. The input x may include a diverse range of categorical and numerical features, extending beyond basic time series information. Unlike conventional time series models that learn directly from data, the target model in this framework does not. Instead, it acts as a fixed structure that imposes an inductive bias, guiding the Hyper-Tree $\mathcal{T}$ to encode temporal dependencies into the parameters it generates. This happens through gradient-based feedback, allowing the Hyper-Tree to generate time- and feature-specific parameters $\hat{\theta}_{\mathcal{T}}(x)$. In this sense, the target model acts more as a function applying the parameters generated by the Hyper-Tree, rather than learning the temporal dependencies on its own. By reformulating well-established GBDTs

in this way, the Hyper-Tree architecture combines the strengths of tree-based learning of target-model parameters with classical time series models, thereby extending GBDTs to a time series context more naturally. Notably, because Hyper-Trees operate in parameter space by learning the parameters of a target time series model, which then creates the final forecasts, they can extrapolate beyond the value range of the training data, overcoming a key limitation of standard tree-based forecasting.

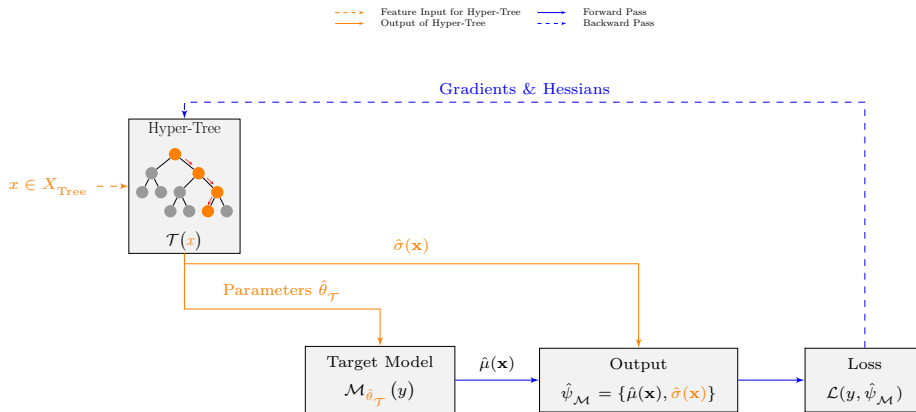
The modularity of our framework enables researchers and practitioners to easily customize the Hyper-Tree framework to their specific contexts, including probabilistic extensions where, instead of $\hat{\psi}_{\mathcal{M}}$ being a point-forecast, the Hyper-Tree can be used to model and forecast parameters of a distribution, from which forecast intervals can be derived.³ In general, any suitable distribution that characterizes the data-generating process can be parameterized. Assuming, e.g., a Normal distribution $y_t \mid \mathbf{x}_t \sim N(\mu_t(\mathbf{x}_t), \sigma_t(\mathbf{x}_t))$, the mean and standard deviation can be specified as

$$\mu_t(\mathbf{x}_t) = \sum_{j=1}^p \theta_{j,t}^{\mu}(\mathbf{x}_t) y_{t-j} \quad (9)$$

$$\sigma_t(\mathbf{x}_t) = \exp(\theta_t^{\sigma}(\mathbf{x}_t)) \quad (10)$$

In this example, we parameterize $\mu_t(\mathbf{x}_t)$ via an AR(p) model. While we can impose temporal structure on $\sigma_t(\mathbf{x}_t)$ as well, e.g., ARCH/GARCH models (Engle, 1982; Bollerslev, 1986), one can alternatively estimate $\sigma_t(\mathbf{x}_t)$ directly via the Hyper-Tree without using an intermediate target model as illustrated in Figure 3. Instead of modeling parameters of a distribution, creating probabilistic forecasts via conformal predictive distributions (Vovk et al., 2022; Johansson et al., 2023) or conformalized quantile regression (Romano et al., 2019) present interesting alternatives. For the scope of this paper, however, we focus on point forecasts, leaving probabilistic extensions for future research.

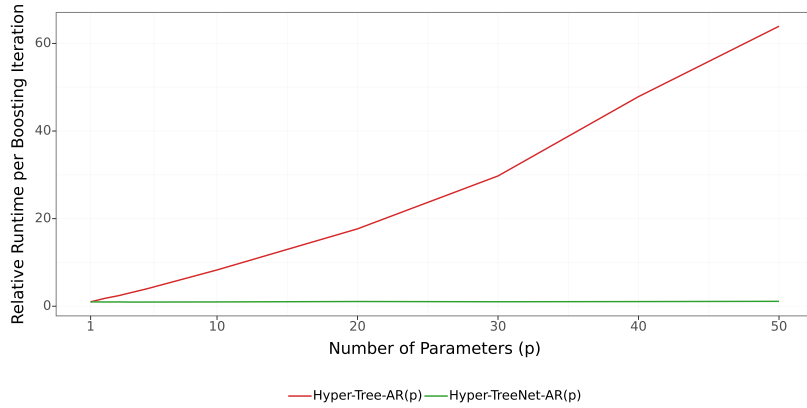
Figure 3: Distributional Hyper-Tree Architecture for a probabilistic framework, where the Hyper-Tree generates parameters for both a target model and an output distribution. The target model outputs the estimated mean $\hat{\mu}(\mathbf{x})$ while the Hyper-Tree directly estimates the standard deviation $\hat{\sigma}(\mathbf{x})$, enabling probabilistic forecasting.



³For extensions of tree-based models to a probabilistic setting, we refer the reader to März & Kneib (2022); März (2022, 2019); Sprangers et al. (2021); Hasson et al. (2021); Duan et al. (2020).

To maintain computational efficiency, we propose a hybrid architecture that integrates GBDTs with neural networks, thereby addressing the scalability limitations of GBDTs when estimating a large number of target model parameters. We refer to this model as Hyper-TreeNet. Traditionally, GBDTs employ a one-vs-all strategy, growing a separate tree for each parameter. While efficient for a small number of parameters, this approach becomes computationally prohibitive as the number of target model parameters increases (Iosipoi & Vakhrushev, 2022). The scaling issue is illustrated in Figure 4, where we plot the relative runtime per boosting iteration for a Hyper-Tree-AR(p) model trained with the one-vs-all strategy, along with the proposed Hyper-TreeNet-AR(p) architecture.

Figure 4: Scaling performance comparison between Hyper-Tree-AR(p) and Hyper-TreeNet-AR(p) models. The figure shows runtimes as the number of AR-parameters increases. All runtimes are normalized with respect to the runtime of estimating one target model parameter.



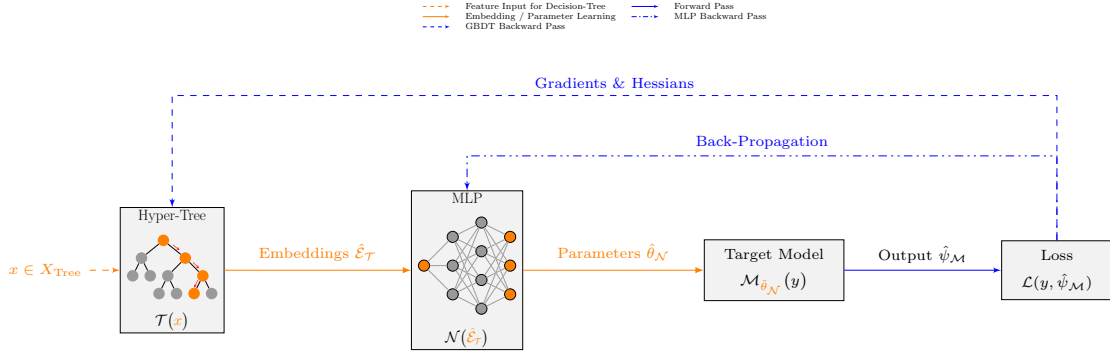
Clearly, the Hyper-TreeNet-AR(p) model maintains a nearly constant runtime regardless of the number of parameters. Our hybrid approach leverages GBDTs to generate low-dimensional embeddings, which are then transformed by a random projection matrix before serving as input to a shallow neural network, typically an MLP (Rumelhart et al., 1986). The Hyper-TreeNet-AR(p) architecture illustrated in Figure 5 resembles an encoder-decoder framework, where the GBDT part acts as an encoder for the input features, and the MLP functions as a decoder, mapping the tree-embeddings to the target model parameters. Notably, the GBDT component generates embeddings in parameter rather than forecasting space. This ensures that both the tree embeddings and network outputs operate within the same space, facilitating more effective learning of time-varying parameters.

For all experiments, we use a one-dimensional tree-embedding to keep the runtime low.⁴ To compensate for this low dimensionality and to enhance the learning capacity of the network, we incorporate a random projection matrix $\mathbf{W}_{\text{proj}}$ following the implementation of Yeh et al. (2024). This expansion diversifies the tree-representations and increases the input dimensions for the subsequent neural network without increasing computational complexity (Yeh et al., 2024; Iosipoi & Vakhrushev, 2022; Casale et al., 2011). Specifically, we generate a matrix $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{k \times d}$ where d is the dimension of tree embeddings and k is the target dimension. Each element of $\mathbf{W}_{\text{proj}}$ is sampled

⁴While $d = 1$ may appear restrictive, our analyses in Appendix G indicate that low-dimensional tree-embeddings yield competitive accuracy on most datasets.

from a standard Normal distribution. The projection operation can be expressed as $\mathbf{z} = \mathbf{W}_{\text{proj}} \mathcal{E}_{\mathcal{T}}$, where $\mathcal{E}_{\mathcal{T}} \in \mathbb{R}^{d \times N}$ represents the tree embeddings for all $i = 1, \dots, N$ observations and $\mathbf{z} \in \mathbb{R}^{k \times N}$ represents the expanded representation passed to subsequent layers of the MLP. From an implementation perspective, $\mathbf{W}_{\text{proj}}$ is a linear network layer with weights that are randomly initialized and then fixed during training, ensuring they remain constant during backpropagation. While higher-dimensional tree embeddings can be used as well, this comes at the cost of increased runtime. Our experiments indicate that a one-dimensional embedding combined with the random projection matrix offers a good trade-off between accuracy and computational efficiency. For all of our experiments, we use the tree-embeddings as the sole input to the MLP, rather than also including the original features, leveraging the GBDT’s ability to effectively encode both categorical and numeric features. Our choice of an MLP is primarily motivated by its lightweight architecture and by evidence in the time series literature showing that MLP-type models are competitive with more complex architectures (Zeng et al., 2023; Chen et al., 2023; Ekambaram et al., 2024).

Figure 5: Hyper-TreeNet architecture illustrating a hybrid approach combining GBDTs and neural networks. A Hyper-Tree generates low-dimensional parameter-space embeddings, which are transformed by a Multi-Layer Perceptron (MLP) to generate parameters for a target model. The architecture allows for joint optimization, enabling integrated tree-based feature learning and network-based parameter mapping.



Both the GBDTs and the MLP are trained jointly, rather than in a two-step procedure. This joint learning enables GBDTs to create informative low-dimensional representations that neural networks can efficiently transform into high-dimensional parameter vectors. Throughout our experiments, we employ a shallow MLP structure following the sequence **{Tree Embeddings $\rightarrow$ Random Projection $\rightarrow$ Hidden-Layer $\rightarrow$ ReLU $\rightarrow$ Output-Layer $\rightarrow$ Dropout}**.⁵ We maintain a fixed hidden dimension, dropout-rate, and learning rate for the MLPs across all datasets and experiments. We set the projection matrix dimensions $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{k \times d}$ with input dimension $d = 1$ to match our one-dimensional tree embeddings, and output dimension k equal to the number of target model parameters. We experimented with various batching strategies for the Hyper-TreeNet model and found that training both components on the full dataset yields the best results. This is likely because GBDTs capture global patterns more effectively when exposed to all observations, producing more informative tree embeddings.

⁵Placing dropout after the output layer randomly zeros individual AR coefficients during training, acting as parameter regularization.

Integrating GBDTs and neural networks in our Hyper-TreeNet architecture requires careful consideration of how gradients are propagated during training. GBDTs operate in a discrete optimization space with threshold-based splits, whereas neural networks rely on continuous gradient-based updates. We consider two strategies: a fully shared gradient flow (*Option 1*) and a separated approach with per-iteration updates but distinct gradient flows (*Option 2*). While *Option 1* allows for joint optimization through a single backpropagation pass, it may limit the ability of each component to maintain its characteristic learning behavior. *Option 2*, which is used as the default in our experiments, preserves the concept of integrated training between both components but avoids neural network gradients interfering with GBDT updates. The backward flows in Figure 5 use different dash patterns to indicate this separation. We empirically compare both options in our ablation studies (Section 4.3.1), as the impact of gradient flow choice may vary across datasets and become more pronounced under specific conditions. More details on the two gradient flow options, including pseudocode and further analysis, are provided in Appendix E.

3. Related Work

Recent advancements in the application of hyper-networks have yielded considerable contributions across various fields, reflecting an increasing interest. In the area of time series forecasting, Lee et al. (2023) propose HyperGPA, a hyper-network designed to address the challenges of temporal drifts in time series data by generating time-varying parameters, yielding marked improvements in forecast accuracy. Duan et al. (2022) introduce Hyper Time Series Forecasting (HTSF), a hyper-network-based framework that tackles distribution shift problems. HTSF jointly learns time-varying distributions and corresponding forecasting models, demonstrating state-of-the-art performance on several benchmarks. Fons et al. (2022) explore the use of hyper-networks in Implicit Neural Representations (INRs) for time series data representation and analysis. Their work demonstrates the potential of INRs in time series imputation and generative tasks, showcasing competitive performance against existing approaches. To bridge classical and deep learning approaches, Rangapuram et al. (2018) propose using RNNs to learn the parameters of linear state space models for each time series, combining the interpretability of state space formulations with the expressiveness of neural networks. Early work in this direction includes Medeiros & Veiga (2000), who introduce a neural coefficient smooth transition autoregressive model. Their approach employs a linear model with time-varying parameters controlled by a neural network, offering advantages over traditional approaches through the incorporation of linear multivariate thresholds and smooth transitions between regimes. In a related approach, Smyl (2020) introduces a hybrid method that integrates exponential smoothing with recurrent neural networks and demonstrates its effectiveness by winning the M4 forecasting competition (Makridakis et al., 2018). In the econometrics literature, Primiceri (2005) introduces Time-Varying Structural Vector Autoregressions, which allow both coefficients and the variance-covariance matrix of innovations in vector autoregressive models to evolve over time. The study shows how time-varying parameters can capture changing economic relationships, particularly in monetary policy analysis where the transmission mechanisms between interest rates, inflation, and output may shift across different time periods.⁶

⁶Early seminal work on time-varying parameter models includes Belsley (1973b,a); Cooley & Prescott (1973, 1976).

Representing model parameters as functions of features has a counterpart in the statistics literature in the form of varying-coefficient models (VCMs) (Hastie & Tibshirani, 1993), where the coefficients are modeled as functions of covariates. In the time series literature, functional-coefficient autoregressive models (Chen & Tsay, 1993; Cai et al., 2000) estimate autoregressive coefficients as smooth functions of an index variable using kernel and local-linear methods. Along these lines, Yousuf & Ng (2021) estimate the coefficients of a high-dimensional predictive regression as smooth functions of time through L_2 boosting with componentwise local-constant or local-linear base learners, and establish consistency under local stationarity, noting that regression trees can serve as alternative base learners. On the tree-based side, Wang & Hastie (2014) use vector-valued trees in a boosted VCM for product-demand prediction, Zhou & Hooker (2022) grow per-coefficient gradient-boosted ensembles with an accompanying consistency guarantee, and Berger et al. (2019) and Bürgin & Ritschard (2017) employ single recursive-partitioning trees, while Puth et al. (2020) extend the former to time-varying coefficients in discrete time-to-event models. More recently, Zakrisson & Lindholm (2025) model the varying coefficients with a cyclic gradient boosting machine (DeLong et al., 2023), and Spuck et al. (2025) develop selective-inference confidence intervals for tree-structured varying coefficients.

To harness the complementary strengths of tree-based methods and neural networks, there has been growing interest in hybrid models that integrate the two frameworks. In their approaches, Karingula et al. (2022) and Badirli et al. (2020) develop variants of gradient boosting that use deep neural networks as weak learners. In a parallel line of research, Emami & Martínez-Muñoz (2022) explore training the final layers of a neural network using gradient boosting, whereas Kontschieder et al. (2016) propose integrating differentiable decision trees into neural networks for end-to-end training. Building on and generalizing the work of Prokhorenkova et al. (2018), Popov et al. (2020) introduce NODE, a deep learning architecture for tabular data that reformulates oblivious decision tree ensembles as differentiable layers. NODE supports end-to-end gradient-based optimization and hierarchical representation learning, enabling integration into standard deep learning frameworks and allowing for multi-layer architectures akin to deep, end-to-end trained GBDTs. Related to this is the approach of Horrell (2025) that implements GBDTs as layers within the PyTorch framework of Paszke et al. (2019). For a more exhaustive overview of combining GBDTs with deep learning, we refer to Li et al. (2023).

4. Experiments

For our experiments, we use the following commonly used publicly available datasets:

- **Air Passengers:** This dataset contains the monthly totals of international airline passengers and is taken from Alexandrov et al. (2020); Athanasopoulos et al. (2011).
- **Australian Electricity Demand:** This dataset comprises five time series, each representing the electricity demand for five Australian states, and is borrowed from Godahewa et al. (2021b). Originally recorded at 30-minute intervals, we aggregate the data to a monthly time granularity for our analysis. This aggregation results in some months with incomplete data (when the original 30-minute observations do not span the entire month), which we exclude from the test set to ensure evaluation on complete months only.

- **Australian Retail Turnover:** This dataset consists of a total of 133 monthly series and is borrowed from [O’Hara-Wild et al. \(2022\)](#).
- **M3 Monthly:** This dataset comprises 1,428 monthly time series, belonging to six distinct domains: demographic, microeconomic, macroeconomic, industry, finance, and other categories, and is borrowed from [Makridakis & Hibon \(2000\)](#); [Godaheva et al. \(2021b\)](#).
- **M3 Yearly:** This dataset comprises 645 yearly time series, belonging to six distinct domains: demographic, microeconomic, macroeconomic, industry, finance, and other categories, and is borrowed from [Makridakis & Hibon \(2000\)](#); [Godaheva et al. \(2021b\)](#).
- **M5:** This dataset comprises daily sales records from Walmart, the world’s largest retailer by revenue. It encompasses 3,049 products across multiple stores and states, resulting in 42,840 distinct time series due to its multi-level hierarchical structure ([Makridakis et al., 2022a,b](#)). We aggregate the data to the store-department level, yielding 70 daily time series. This aggregation is motivated primarily to reduce the intermittency present at the most granular level of the data.
- **Rossmann Store Sales:** This dataset consists of a total of 1,115 daily series and is borrowed from [Knauer & Cukierski \(2015\)](#). We subset the data to include only stores that are open.
- **Tourism:** This dataset consists of a total of 366 monthly series and is borrowed from [Alexandrov et al. \(2020\)](#); [Athanasopoulos et al. \(2011\)](#).

As references, we compare our Hyper-Trees to the following models:

- **AR(p):** An autoregressive model with p lags.
- **AR(p)-X:** same as AR(p) with additional features.
- **AutoArima:** An ARIMA ([Box et al., 2015](#)) model where optimal $(p, d, q)(P, D, Q)$ are selected.
- **AutoArima-X:** same as AutoArima with additional features.
- **AutoETS:** An Exponential Smoothing ([Holt, 1957](#); [Winters, 1960](#)) model, where the model specification is selected automatically.
- **Chronos:** Chronos is a family of pre-trained forecasting models based on language model architectures ([Ansari et al., 2024](#)). For all experiments, we use the **chronos-t5-base** version.
- **Deep-AR:** A probabilistic forecasting model that uses autoregressive recurrent neural networks ([Salinas et al., 2020](#)).
- **LightGBM:** LightGBM model of [Ke et al. \(2017\)](#).
- **LightGBM-AR(p):** Same as LightGBM with additional autoregressive lagged-target values of order p as features.
- **LightGBM-STL:** Each series is first de-trended and de-seasonalized via a cubic polynomial. The remainder is then modeled via a local LightGBM ([Ke et al., 2017](#)) model.
- **Temporal Fusion Transformer (TFT):** An attention-based architecture for multi-horizon time series forecasting ([Lim et al., 2021](#)).

Classical time series models are trained locally on each series and evaluated on datasets where only time-derived features (e.g., month, quarter) are available. Due to their cross-learning capabilities, as well as their ability to incorporate a richer feature set including categorical and

numeric variables alongside temporal information, deep-learning-based models are used for global model training and evaluation only. All local forecasting models are trained using the Nixtla-implementation of Garza et al. (2022), whereas the deep-learning models are trained via the implementations available in GluonTS of Alexandrov et al. (2020); Athanasopoulos et al. (2011). The LightGBM-STL model is trained using the sktime implementation of Király et al. (2024); Löning et al. (2019). To ensure a practical comparison across models, we standardized hyper-parameters as much as possible across all models for each specific dataset. Following real-world practices where exhaustive tuning is often impractical due to time or computational constraints, we opted not to perform extensive hyper-parameter optimization and used mostly default configurations. We refer to Tables C1 and C2 for details on the hyper-parameters. It is also important to note that, unlike conventional tree-based time series models, Hyper-Trees exclude all target-derived features, including lags, moving averages, and other transformations of the target variable, from their feature set, relying instead on the parameterized target time series model to capture temporal dynamics. For our experiments, we start with a simple illustrative example in Section 4.1 and continue with an evaluation across multiple datasets in Sections 4.2 and 4.3. Owing to its proven accuracy and popularity in the time series community, we use LightGBM (Ke et al., 2017) as the GBDT for our Hyper-Tree models across all experiments.

4.1. Local Time Series Model: Introductory Example

We start our analysis with the **Air Passengers** dataset, which consists of monthly totals of airline passengers from 1949 to 1960. It is a popular dataset in time series forecasting and its trend and pronounced seasonality characteristics make it particularly suitable for the ARIMA (Box et al., 2015) to be a strong baseline. We split the data into train and test, where the last year is used as a hold-out set. The first step for analyzing such a time series dataset is typically a decomposition into seasonality, trend, and remainder. This STL decomposition is usually done via a LOESS-based estimation of each component (Cleveland et al., 1990). However, in the following, we use our Hyper-Tree architecture for decomposing the train series. The **Hyper-Tree-STL** parameterizes the following trend and seasonality components

$$\text{Trend}_t = a_{0,t}(\mathbf{x}_t) + a_{1,t}(\mathbf{x}_t) \cdot t \quad (11)$$

$$\text{Seasonality}_t = \sum_{r=1}^{N_{\text{season}}} \left(c_{r,t}(\mathbf{x}_t) \cdot \sin \left(t \cdot r \cdot \frac{2\pi}{m_{\text{season}}} \right) + d_{r,t}(\mathbf{x}_t) \cdot \cos \left(t \cdot r \cdot \frac{2\pi}{m_{\text{season}}} \right) \right) \quad (12)$$

where the parameters $\theta_T(\mathbf{x}_t) = [a_{0,t}(\mathbf{x}_t), a_{1,t}(\mathbf{x}_t), \{c_{r,t}(\mathbf{x}_t), d_{r,t}(\mathbf{x}_t)\}_{r=1}^{N_{\text{season}}}]$ are modeled as functions of time-derived features $\mathbf{x}_t = \{\text{month}, \text{quarter}, \text{year}, \text{time}\}$, with time being a linearly increasing integer. Even though the trend part is specified as a linear function of t , the interactions and non-linearities induced by the Hyper-Tree allow for modeling complex patterns. To ensure the trend is a smooth function of time, we add a term that penalizes squared first and second order differences, thereby encouraging neighboring estimates to be close to each other. We represent the seasonality via periodic Fourier-terms that are composed of sine and cosine components, each with different frequencies and amplitudes, where $\{c_{r,t}(\mathbf{x}_t), d_{r,t}(\mathbf{x}_t)\}_{r=1}^{N_{\text{season}}}$ represent the sine and cosine weights for the r -th seasonal component. For this experiment, we set $N_{\text{season}} = 1$ and $m_{\text{season}} = 12$ since the data is recorded on a monthly frequency. Instead of tuning the hyper-parameters, we

mostly use default values and the MSE as the loss function. Figure 6 shows the trend and seasonality estimated on the training dataset. As a reference, we add the components of a conventional STL decomposition as available in [Seabold & Perktold \(2010\)](#).

Figure 6: **Hyper-Tree-STL** based Decomposition.

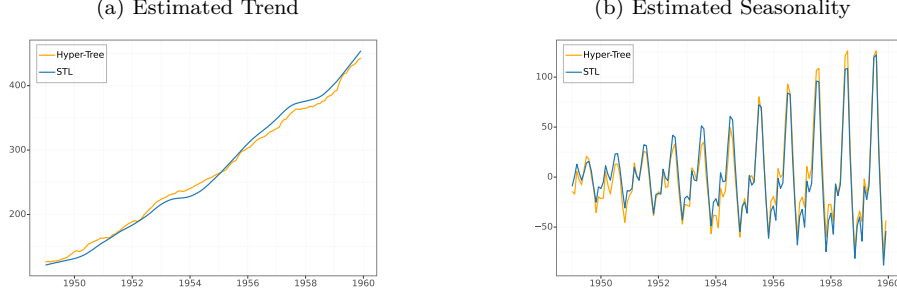
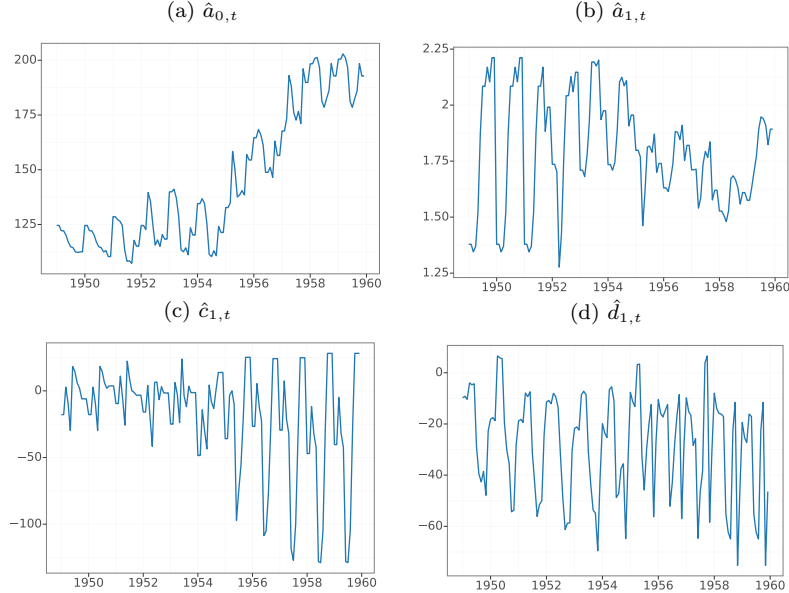


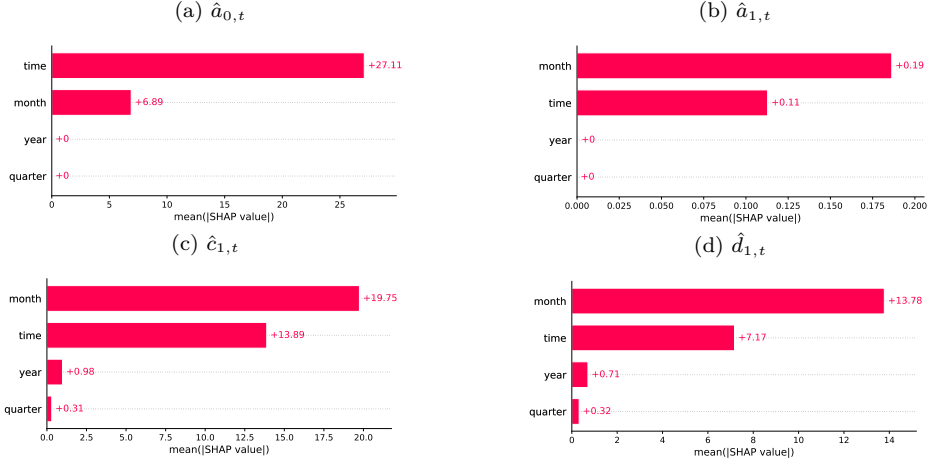
Figure 6 shows that the STL components estimated via the **Hyper-Tree-STL** and the traditional LOESS-based decomposition are in close agreement, even though the trend-estimate of the **Hyper-Tree-STL** is not as smooth. Considering the current use of relatively simplistic elements for estimating trend and seasonality, their approximation quality could be enhanced by using more complex target models, e.g., via penalized splines ([Wood, 2017](#)). In terms of model interpretability, we can visualize the estimated parameters over time, as illustrated in Figure 7.

Figure 7: Estimated Parameters of **Hyper-Tree-STL**.



Besides the estimated parameters, we also have access to SHAP values ([Lundberg et al., 2020](#); [Lundberg & Lee, 2017](#)) and feature importances for each estimated parameter, further increasing model interpretability.

Figure 8: Feature Importance of Hyper-Tree-STL Parameters.



The feature importances for the trend parameters (Panels 8a and 8b) show that mostly features related to the progression of time are considered important, while the month-feature that reflects the within year patterns ranks highest for the seasonality parameters (Panels 8c and 8d). From the above STL discussion, we can infer that our Hyper-Tree-STL approach gives results close to the conventional decomposition, while providing the additional capabilities of adaptive tree-based parameter estimation and enhanced interpretability.

As the second step of our analysis, we now turn to the forecasting part for the last 12 months of the *Air Passengers* dataset. As target models, we specify the following Hyper-Tree-AR(12) and Hyper-TreeNet-AR(12) models

$$y_t = \theta_{1,t}(\mathbf{x}_t)y_{t-1} + \theta_{2,t}(\mathbf{x}_t)y_{t-2} + \dots + \theta_{12,t}(\mathbf{x}_t)y_{t-12} \quad (13)$$

with time-step and feature specific parameters $\{\theta_{1,t}(\mathbf{x}_t), \dots, \theta_{12,t}(\mathbf{x}_t)\}$, as well as an Hyper-Tree-ETS model with a damped trend and multiplicative seasonality (Hyndman & Athanasopoulos, 2021)

$$\ell_t = \alpha_t(\mathbf{x}_t) \frac{y_t}{s_{t-m_{\text{season}}}} + (1 - \alpha_t(\mathbf{x}_t))(\ell_{t-1} + \phi_t(\mathbf{x}_t)b_{t-1}) \quad (14)$$

$$b_t = \beta_t(\mathbf{x}_t)(\ell_t - \ell_{t-1}) + (1 - \beta_t(\mathbf{x}_t))\phi_t(\mathbf{x}_t)b_{t-1} \quad (15)$$

$$s_t = \gamma_t(\mathbf{x}_t) \frac{y_t}{(\ell_{t-1} + \phi_t(\mathbf{x}_t)b_{t-1})} + (1 - \gamma_t(\mathbf{x}_t))s_{t-m_{\text{season}}} \quad (16)$$

where $\{\ell_t, b_t, s_t\}$ denote {level, trend, seasonality} with corresponding time-step and feature specific parameters $\theta_{\mathcal{T}}(\mathbf{x}_t) = [\alpha_t(\mathbf{x}_t), \beta_t(\mathbf{x}_t), \gamma_t(\mathbf{x}_t), \phi_t(\mathbf{x}_t)]$, where m_{season} indicates the frequency of the seasonality. The h-step ahead forecast of the Hyper-Tree-ETS model is given by

$$\hat{y}_{t+h|t} = \left[\hat{\ell}_t + \sum_{u=1}^h \left(\prod_{v=1}^u \hat{\phi}_{t+v}(\mathbf{x}_{t+v}) \right) \hat{b}_t \right] \hat{s}_{t+h-m_{\text{season}}(k_{\text{season}}+1)} \quad (17)$$

The choice of the target AR(12) models is not based on automated model selection. Instead, the decision to use 12 lags stems from the monthly nature of the dataset. Similarly, the trend

and seasonality of the `Air Passengers` dataset have led us to use Exponential Smoothing (Holt, 1957; Winters, 1960) with multiplicative seasonality and damped trend as the target model for the `Hyper-Tree-ETS`, since this specification often provides accurate and robust forecasts (Hyndman & Athanasopoulos, 2021).

Before we turn to the results presented in Table 1, it is important to note that the `Hyper-Tree-ETS` model differs structurally from all other `Hyper-Tree` variants in a way that has direct consequences for the Hessian computation. In the AR-model variants, the fitted value $\hat{y}_t$ depends only on the parameters at time t and the corresponding lags, making the Hessian block-diagonal across time steps and the standard GBDT second-order computation numerically stable. The ETS model, by contrast, is a state-space recurrence: equations (14)-(16) show that each state $\{\ell_t, b_t, s_t\}$ depends on the smoothing parameters at all preceding time steps. The resulting Hessian is therefore dense rather than block-diagonal, and its cross-temporal second derivatives vanish or explode through long recurrences, analogous to second-order vanishing/exploding gradients in RNNs (Bengio et al., 1994; Pascanu et al., 2013), rendering the standard GBDT curvature estimates numerically unstable. We therefore approximate the Hessian using the Gauss-Newton method (Nocedal & Wright, 2006; Martens, 2020), which drops the residual-curvature term, whose second-order derivatives propagate through the full recurrence, retaining only the first-order (Jacobian) term and guaranteeing positive semi-definite and numerically stable curvature estimates. The diagonal of the resulting approximation is estimated via Hutchinson’s stochastic diagonal estimator (Hutchinson, 1990) with K random Gaussian probes.⁷

Table 1: `Air Passengers` Results.

Model		MAPE	sMAPE	WAPE	RMSE	MAE
AR		8.630	9.206	9.302	54.891	44.292
AR-X		8.706	9.292	9.354	54.995	44.542
AutoARIMA		4.180	4.031	3.889	23.919	18.516
AutoARIMA-X		3.294	3.200	3.080	19.525	14.665
AutoETS		4.499	4.589	4.677	25.280	22.270
Hyper-Tree-AR		2.524	2.470	2.395	15.783	11.406
Hyper-Tree-ETS		3.899	3.965	3.750	20.469	17.858
Hyper-TreeNet-AR		4.119	4.144	3.972	21.289	18.915
LightGBM		3.070	3.068	2.927	17.765	13.939
LightGBM-AR		3.752	3.699	3.620	20.973	17.238
LightGBM-STL		3.796	3.822	3.923	23.471	18.678

Reported are the forecast errors, with lower values indicating better performance and the best metrics highlighted in bold. Mean Absolute Percentage Error (MAPE); Symmetric Mean Absolute Percentage Error (sMAPE); Weighted Absolute Percentage Error (WAPE); Root Mean Squared Error (RMSE); Mean Absolute Error (MAE).

The results in Table 1 show that `Hyper-Tree` models are competitive, with `Hyper-Tree-AR` being the most accurate across all evaluated metrics. Despite sharing the same underlying autoregressive structure but with static parameters, `AR-X` exhibits the lowest accuracy across all metrics.

⁷While gradients also propagate through the recurrence, they involve only products of Jacobians, which are stabilized by the contractive ETS state equations, whose smoothing parameters are bounded in $(0, 1)$. The Hessian is more severely affected because the residual-curvature term requires second-order derivatives of the full recurrence, making the computation numerically less stable than the gradients. The Gauss-Newton approximation of the Hessian eliminates this term, retaining only the Jacobian-based component that shares the stability properties of the gradients.

The **Hyper-Tree-AR** appears to capture the behavior of the series more effectively, likely due to its time-varying AR-parameters that are modeled as a function of time-related features and their interactions. While **AutoARIMA** benefits from automatic model selection and differencing, the selected model is an $\text{ARIMA}(1,1,0)(0,1,0)$ [12], our **Hyper-Tree-AR** achieves competitive results with a plain $\text{AR}(12)$ specification without differencing or automated lag selection. Hence, the accuracy of the **Hyper-Tree-AR** model could likely be improved by incorporating seasonal differencing or by adapting the lag structure via automatic selection. Among the other variants, **Hyper-TreeNet-AR** and **Hyper-Tree-ETS** models compare well across metrics, though neither matches the accuracy of the **Hyper-Tree-AR**. Yet, **Hyper-Tree-ETS** yields more accurate results than its classical counterpart **AutoETS** across all metrics and remains competitive with both **AutoARIMA** variants. Conventional tree-based models, while performing well, lag behind our **Hyper-Tree-AR** model.

4.2. Local Time Series Models: Extended Evaluation

In the previous section, we evaluated the Hyper-Tree architectures on a single, well-behaved time series with strong seasonality and trend characteristics that can easily be modeled using time-derived features. To assess the accuracy of our Hyper-Tree architectures more comprehensively, we now extend our evaluation to additional datasets with more diverse characteristics. For the **Australian Retail Turnover** and **Tourism** datasets, we modify our **Hyper-Tree-ETS** training approach due to computational constraints. The recursive nature of ETS model training requires iterating over the entire time series at each boosting step, making per-series training of our **Hyper-Tree-ETS** model computationally expensive for datasets containing many long series. To address this challenge, we leverage a unique property of ETS models: their parameters are bounded in $[0, 1]$, enabling a simplified parameterization. Rather than learning parameters for each series individually, we adopt a parameterization strategy where we set parameter values for all series globally. Specifically, we set $\{\alpha_t, \beta_t, \gamma_t, \phi_t\} = 0.3$ for all series in both datasets, selected through grid search over $\{0.1, \dots, 0.9\}$ on a hold-out set optimizing for WAPE. Table 2 presents the results.

Our evaluation reveals several interesting patterns across the three datasets. Most notably, **Hyper-TreeNet-AR** consistently ranks among the top models. Our hybrid model outperforms both classical time series models and conventional tree-based approaches, demonstrating the effectiveness of combining tree-based representations with neural parameter estimation. The purely tree-based Hyper-Tree variants also show strong performance. On the **Australian Electricity Demand** dataset, **Hyper-Tree-AR** surpasses its classical counterparts (**AR**, **AR-X** and **AutoARIMA-X**) on most metrics. On the **Australian Retail Turnover** dataset, the **Hyper-Tree-ETS** model performs reasonably well, suggesting that exponential smoothing captures the dataset’s seasonal patterns. For some datasets, the standard **LightGBM** model yields more accurate forecasts than the **LightGBM-AR** variant, despite the latter’s use of autoregressive features. This points to the possibility that simply adding lagged target values to tree-based models may not effectively capture temporal dependencies without an appropriate inductive bias, validating our hyper-model approach that naturally integrates time series dynamics into tree-based models.

Table 2: Local Model Results.

Dataset	Model	MAPE	sMAPE	WAPE	RMSE	MAE
Australian Electricity Demand	AR	5.291	5.129	5.151	257,965.5	224,999.9
	AR-X	5.166	5.014	5.035	253,778.7	222,623.4
	AutoARIMA	3.421	3.398	3.430	199,044.3	158,517.8
	AutoARIMA-X	4.146	4.048	4.134	225,535.0	188,831.5
	AutoETS	3.480	3.430	3.479	177,123.1	133,774.5
	Hyper-Tree-AR	4.140	4.047	4.076	218,432.8	193,903.3
	Hyper-Tree-ETS	3.845	3.765	3.800	204,600.1	172,143.5
	Hyper-TreeNet-AR	2.925	2.939	2.938	160,191.1	118,736.9
	LightGBM	3.067	3.076	3.081	178,666.6	137,318.4
	LightGBM-AR	5.232	5.086	5.148	266,541.8	237,194.2
	LightGBM-STL	6.208	6.556	6.273	310,754.9	242,870.6
Australian Retail Turnover	AR	9.090	9.089	9.131	30.694	24.463
	AR-X	8.320	8.309	8.372	27.563	21.716
	AutoARIMA	7.146	7.096	7.051	18.533	16.235
	AutoARIMA-X	6.846	6.681	6.755	18.268	15.731
	AutoETS	6.101	6.113	6.112	17.128	14.037
	Hyper-Tree-AR	6.997	6.781	7.001	19.728	16.795
	Hyper-Tree-ETS	6.312	6.399	6.372	20.062	16.800
	Hyper-TreeNet-AR	6.248	6.218	6.298	16.612	13.659
	LightGBM	9.180	8.864	9.025	20.368	17.166
	LightGBM-AR	9.482	8.908	9.455	23.676	19.736
	LightGBM-STL	10.734	11.020	10.905	26.810	23.162
Tourism	AR	25.052	21.846	20.275	2,979.4	2,291.9
	AR-X	24.927	21.644	19.922	2,924.1	2,238.9
	AutoARIMA	21.132	19.647	18.310	2,624.0	2,092.0
	AutoARIMA-X	20.467	19.086	17.957	2,522.9	1,984.3
	AutoETS	20.880	19.320	18.528	2,735.6	2,164.3
	Hyper-Tree-AR	22.889	18.813	18.159	3,172.8	2,605.0
	Hyper-Tree-ETS	31.648	19.228	26.818	2,595.8	1,954.4
	Hyper-TreeNet-AR	20.216	18.419	17.451	3,239.4	2,690.3
	LightGBM	31.843	25.337	22.550	4,377.8	3,785.0
	LightGBM-AR	30.448	23.504	23.422	4,311.3	3,270.9
	LightGBM-STL	30.529	27.608	27.392	5,047.4	3,852.4

Reported are the mean forecast errors across series per dataset, with lower values indicating better performance and the best metrics highlighted in bold. Mean Absolute Percentage Error (MAPE); Symmetric Mean Absolute Percentage Error (sMAPE); Weighted Absolute Percentage Error (WAPE); Root Mean Squared Error (RMSE); Mean Absolute Error (MAE).

4.3. Global Time Series Models

In previous sections, we evaluated Hyper-Trees in a local forecasting setting, training separate models for each time series individually without leveraging cross-series information. We now shift focus to global forecasting (Januschowski et al., 2020), where Hyper-Trees can leverage their ability to capture shared patterns across multiple series. Operational forecasting (Januschowski & Kolassa, 2019) typically involves datasets with high-dimensional feature sets that go beyond basic time series information, including numeric and categorical features. In such environments, tree-based models typically excel, providing a suitable setting to assess the accuracy of Hyper-Trees. To create a more realistic testing ground for our framework, we add the **Rossmann** and **M5** datasets that closely resemble real-world corporate datasets, in addition to the datasets used in the local forecasting evaluation. Also, we add the commonly used **M3 Monthly** and **M3 Yearly** datasets. Local forecasting models, including **LightGBM-STL**, are excluded from our global forecasting evaluation. This decision stems from their inherent limitations in handling complex, multi-series forecasting tasks. Unlike global models, local models cannot leverage cross-series information, as they are typically trained on individual time series in isolation. This limitation in learning from multiple

series simultaneously hinders their ability to capture broader patterns across the entire dataset, potentially reducing their generalization capabilities (Montero-Manso & Hyndman, 2021). Furthermore, local models struggle to effectively incorporate categorical features, which are crucial for borrowing strength across similar series. Yet, while we exclude local models from our global model evaluation, we use **AutoETS** forecasts as the reference for calculating a scaled error metric following the Mean Absolute Scaled Error (MASE, Hyndman & Koehler (2006)). This choice is based on the strong performance of the **AutoETS** among local models, as reported in Table 2, making it a robust benchmark for assessing the accuracy of global forecasting models.

Besides comparing our approach to conventional tree-based and deep-learning models, we also include a foundation time series model in our evaluation. This inclusion reflects the trend in time series forecasting towards large-scale, pre-trained models. Among others, the most prominent include Chronos (Ansari et al., 2024), Moirai (Woo et al., 2024), Moment (Goswami et al., 2024), TimesFM (Das et al., 2024), Tiny Time Mixers (Ekambaram et al., 2024) and Lag-Llama (Rasul et al., 2023). Trained on extensive and diverse time series datasets, these models aim to improve accuracy and generalization in forecasting tasks. For our experiments, we use Chronos (Ansari et al., 2024), which has demonstrated strong performance across multiple benchmark datasets.⁸ Compared to deep-learning models that learn abstract representations from time series data, it is important to note that tree-based models, including our Hyper-Tree approaches, operate on the information provided to them and cannot generate new features that are not present in the original input. Instead, they excel at modeling complex relationships within the provided feature set. To better capture the overall characteristics of each series and enhance the local adaption of global tree-based models across diverse time series datasets, we incorporate additional features using the established approach of Hyndman et al. (2024), which characterizes each series based on information derived from the original time series. While embeddings from foundation models like Chronos (Ansari et al., 2024) could be used as feature inputs to the tree-models as well, we opted for traditional statistical features to ensure that the evaluation of our Hyper-Tree models is not influenced by the quality of the embeddings of a foundation model.

To ensure as fair a comparison as possible across models, we have made several decisions for our experimental setup. We disable the standard lag and time-feature generation in **Deep-AR** (Salinas et al., 2020) and **TFT** (Lim et al., 2021). Instead, we use the same lag specification and time-derived features across all models. Also, we exclude the time series derived features of Hyndman et al. (2024) from deep-learning models and use them only in tree-based models, as these are not commonly used in neural network architectures. We estimate the 50% quantile using **TFT** (Lim et al., 2021), while for **Deep-AR** (Salinas et al., 2020), we calculate the mean from 1,000 samples drawn from a forecasted Student-T distribution. In the case of **Chronos** (Ansari et al., 2024), we use the **t5-base** version with default parameters and draw 50 samples from the forecasted distribution. We evaluate the **Hyper-Tree-ETS** model only on datasets with stable seasonality across series, in line with the multiplicative seasonality assumption of the ETS specification, which excludes the **M3 Monthly**, **M5**, and **Rossmann** datasets. Further, the vectorized **Hyper-Tree-ETS** estimation requires uniform series lengths, with the data structured in a tensor of shape (**number of series**, **context-length**, **number of channels**). This format allows for efficient vectorized

⁸According to Ansari et al. (2024), Chronos was not pre-trained on any of the datasets used in our evaluation and all forecasts of Chronos (Ansari et al., 2024) reported in Table 3 are zero-shot.

estimation across multiple time series but necessitates that all series have the same length within a given dataset. For datasets with varying series lengths, we employ the following strategy for the **Hyper-Tree-ETS** models: instead of using zeros or constants, we back-append a segment from the series itself to match the length of the longest series. This preserves the series’ characteristics, preventing distortion of seasonality and trend initialization that would occur with constant or zero-padding. To ensure that padded observations do not influence the estimation of ETS parameters, we apply a mask during training that zeroes out the contributions of padded time steps and also add a binary flag as an additional feature to identify padded observations. All other models in our evaluation use the datasets in their original form without this preprocessing step. Additionally, for the **Hyper-Tree-ETS** model, we apply a scaling transformation to each series of the **Tourism** dataset by adding a constant offset value. This transformation is necessary for the multiplicative seasonality specification to avoid zeros and prevent numerical instability. For the **M3 Yearly** dataset, which lacks seasonality and is predominantly driven by trends, we adapt our **Hyper-Tree-ETS** model to an ETS formulation with a linear trend only.

Turning to the results in Table 3, our Hyper-Tree models demonstrate competitive performance across a range of established baselines and datasets. On the **Australian Electricity Demand** dataset, the **Hyper-TreeNet-AR** model leads across all accuracy measures, while on the **Australian Retail Turnover** dataset, **Hyper-Tree-ETS** and **Hyper-TreeNet-AR** share the top positions across metrics. On the operationally more complex **Rossmann Store Sales** dataset, where conventional GBDTs are the typical state-of-the-art, **Hyper-Tree-AR** is the most accurate model. This shows that a classical autoregressive model, when parameterized via GBDTs, can compete with and even surpass conventional tree-based forecasting models, highlighting the competitiveness of our framework in settings where traditional GBDTs typically excel and validating its applicability in complex, feature-rich environments common to operational forecasting. For the **M3 Monthly** dataset, **Chronos** (Ansari et al., 2024) achieves the highest accuracy among all models, albeit with a higher runtime compared to our Hyper-Tree approaches. The **Hyper-Tree-ETS*** is the most accurate model on the **M3 Yearly** data, indicating that exponential smoothing with a linear trend is well-suited for this type of annual data. The **Hyper-Tree-ETS** model with multiplicative seasonality and a damped trend shows good performance on datasets with clear seasonal patterns, though its recursive estimation leads to comparatively higher computational costs. The results of the **Hyper-Tree-ETS** for the **Tourism** dataset, however, indicate that its applicability is limited, likely due to a misalignment between the model’s multiplicative seasonality assumptions and the heterogeneous seasonal patterns in the data. This illustrates a key characteristic of the Hyper-Tree framework: while inductive bias provides advantages when target model assumptions match the data characteristics, accuracy degrades when they diverge. The **M5** dataset results show that while **Deep-AR** achieves the highest accuracy across error metrics, **TFT**, **Chronos** and our **Hyper-TreeNet-AR** remain competitive alternatives, with **Hyper-TreeNet-AR** offering faster runtimes. Table 3 also shows that standard **LightGBM** and **LightGBM-AR** models consistently offer the fastest training times but lag on accuracy measures for some datasets. Overall, these results demonstrate that Hyper-Tree models effectively combine tree-based feature learning with time series inductive biases, bridging traditional statistical approaches and machine learning. The effectiveness of our framework depends on selecting target models whose assumptions align with the underlying data characteristics, making this choice a crucial consideration.

Table 3: Global Model Results.

Dataset	Model	MAPE	sMAPE	WAPE	RMSE	MAE	MASE	Runtime (Minutes)
Australian Electricity Demand	Chronos	3.608	3.579	3.594	189,129.1	138,833.2	1.049	0.027
	Deep-AR	3.594	3.628	3.634	201,459.5	155,123.3	1.142	1.273
	Hyper-Tree-AR	4.125	4.034	4.062	217,635.7	192,981.4	1.300	0.022
	Hyper-Tree-ETS	3.954	3.885	3.920	233,281.0	196,350.0	1.331	0.355
	Hyper-TreeNet-AR	2.964	2.950	2.960	163,026.9	121,972.3	0.886	0.022
	LightGBM	5.909	6.173	5.894	269,868.1	210,619.9	1.723	0.002
	LightGBM-AR	6.086	5.872	5.976	281,027.3	254,324.2	1.842	0.002
	TFT	4.607	4.544	4.594	231,271.9	191,331.3	1.374	2.015
Australian Retail Turnover	Chronos	7.117	6.914	7.026	18.166	15.832	1.243	1.252
	Deep-AR	7.752	7.363	7.685	21.021	18.519	1.403	5.355
	Hyper-Tree-AR	6.877	6.656	6.863	19.613	16.729	1.271	0.618
	Hyper-Tree-ETS	6.202	6.300	6.252	17.966	15.062	1.066	1.166
	Hyper-TreeNet-AR	6.403	6.292	6.427	16.179	13.409	1.090	0.109
	LightGBM	12.950	12.530	12.785	22.006	18.866	2.234	0.012
	LightGBM-AR	8.247	7.882	8.134	21.097	17.926	1.479	0.013
	TFT	9.756	9.085	9.669	26.250	22.725	1.832	10.299
M3-Monthly	Chronos	16.668	13.874	13.724	751.987	618.140	1.138	5.262
	Deep-AR	21.728	14.641	14.739	814.384	673.038	1.342	3.511
	Hyper-Tree-AR	19.580	14.565	14.720	809.820	672.991	1.261	0.401
	Hyper-Tree-ETS	-	-	-	-	-	-	-
	Hyper-TreeNet-AR	19.778	14.659	14.603	814.225	677.430	1.374	0.075
	LightGBM	26.486	17.504	18.578	945.055	803.409	2.391	0.031
	LightGBM-AR	21.445	14.479	14.743	786.295	654.822	1.406	0.029
	TFT	19.752	14.657	14.551	796.385	653.173	1.231	4.021
M3-Yearly	Chronos	20.217	18.306	17.410	1,245.9	1,073.0	1.561	0.893
	Deep-AR	28.425	19.340	22.664	1,447.8	1,236.0	2.741	1.383
	Hyper-Tree-AR	24.631	19.634	20.925	1,647.4	1,424.9	1.839	0.021
	Hyper-Tree-ETS*	21.242	16.650	17.082	1,229.5	1,069.7	1.204	0.130
	Hyper-TreeNet-AR	21.782	18.389	18.133	1,233.0	1,088.0	1.558	0.031
	LightGBM	30.376	23.676	26.071	1,605.0	1,402.6	4.164	0.003
	LightGBM-AR	24.798	17.554	20.522	1,781.6	1,510.1	1.870	0.004
	TFT	27.197	18.603	20.682	1,372.6	1,195.8	1.488	1.912
M5	Chronos	13.997	13.326	13.274	81.722	63.557	0.971	0.875
	Deep-AR	13.231	12.712	12.605	74.547	59.611	0.934	1.463
	Hyper-Tree-AR	17.583	14.918	15.608	91.369	74.621	1.120	0.237
	Hyper-Tree-ETS	-	-	-	-	-	-	-
	Hyper-TreeNet-AR	15.290	13.671	13.926	79.713	63.052	0.990	0.049
	LightGBM	20.469	16.166	18.080	78.348	63.039	1.150	0.012
	LightGBM-AR	15.571	13.910	14.174	87.723	70.362	1.054	0.015
	TFT	14.106	13.045	13.100	79.366	63.579	0.961	2.301
Rossmann Store Sales	Chronos	12.799	12.260	12.284	1,140.4	857.111	0.487	20.137
	Deep-AR	9.897	9.342	9.609	860.314	665.007	0.383	8.482
	Hyper-Tree-AR	8.830	9.023	8.881	798.023	622.134	0.358	8.880
	Hyper-Tree-ETS	-	-	-	-	-	-	-
	Hyper-TreeNet-AR	9.437	9.597	9.449	857.004	664.496	0.385	0.693
	LightGBM	10.456	10.559	10.352	895.679	712.080	0.423	0.155
	LightGBM-AR	9.187	8.898	8.997	806.228	622.404	0.359	0.189
	TFT	9.258	8.913	8.982	822.939	624.228	0.357	11.328
Tourism	Chronos	22.113	23.925	21.739	2,870.8	2,318.0	1.211	2.420
	Deep-AR	23.926	20.408	20.076	2,760.9	2,234.0	1.178	5.808
	Hyper-Tree-AR	22.525	19.375	18.618	3,059.3	2,509.5	1.047	0.855
	Hyper-Tree-ETS	28.638	25.991	22.431	4,501.9	3,504.2	1.278	0.986
	Hyper-TreeNet-AR	23.731	20.603	19.587	3,031.1	2,515.8	1.113	0.135
	LightGBM	47.469	63.132	47.401	9,715.2	9,055.1	3.440	0.030
	LightGBM-AR	27.392	21.774	21.157	4,360.4	3,450.6	1.258	0.029
	TFT	25.617	21.950	21.409	3,476.8	2,848.5	1.270	10.424

Reported are the mean forecast errors across series per dataset and total runtimes in minutes, with lower values indicating better performance and the best metrics highlighted in bold. Mean Absolute Percentage Error (MAPE); Symmetric Mean Absolute Percentage Error (sMAPE); Weighted Absolute Percentage Error (WAPE); Root Mean Squared Error (RMSE); Mean Absolute Error (MAE); Mean Absolute Scaled Error (MASE), which is calculated relative to the AutoETS forecasts (MAE of model / MAE of AutoETS). Entries with '-' indicate that the model was not trained. For the M3 Yearly dataset, the Hyper-Tree-ETS parameterizes an ETS model with a linear trend only, as indicated by '*'. For all other applicable datasets, the Hyper-Tree-ETS generates parameters for ETS models with multiplicative seasonality and a damped trend.

4.3.1. Ablation Studies

In this section, we conduct a series of ablation studies. By systematically removing or modifying components, we aim to gain a better understanding of the contribution of each model element to overall accuracy for the **Hyper-TreeNet-AR** and **Hyper-Tree-AR** models. Our ablations are conducted on the **Rossmann Store Sales** dataset, which closely reflects practical forecasting scenarios in both the scale of available time series and the complexity of the underlying data and features. This provides a suitable environment for evaluating the sensitivity of our models to various components. For our ablation studies, we have chosen the following modifications:

- A1: Increase the dimension of tree-embeddings d from 1 to 5.
- A2: Disable linear-tree option of [Shi et al. \(2019\)](#).
- A3: Increase hidden-dimension of the MLP from 128 to 256.
- A4: Remove random projection layer.
- A5: Reduce the number of lags from 21 to 14.
- A6: Remove *tsfeatures* ([Hyndman et al., 2024](#)) from the set of features.
- A7: Exclude GBDTs as encoding layer and use MLP only.
- A8: Use **TreeNet** to forecast directly, without learning the parameters of a target model.
- A9: Following the two-step approach described by [He et al. \(2014\)](#), we first train a LightGBM model ([Ke et al., 2017](#)) and extract the leaf indices from the decision trees. These leaf indices are then used as input features to a separately trained MLP.
- A10: Train model with a shared gradient flow according to *Option 1* as described in Table [E1](#).
- A11: Instead of using time-varying parameters, we average the parameters across the forecast horizon and use constant AR-parameters for generating forecasts.

Each variant (A1 - A11) is derived from the **Base** model as reported in Table [3](#), with a specific modification applied to create each variation. This approach allows us to isolate and evaluate the impact of individual components. Table [4](#) shows the results. We first analyze the impact of each ablation on **Hyper-TreeNet-AR**, followed by **Hyper-Tree-AR**.

Increasing the dimension of tree embeddings from 1 to 5 (A1) leads to an improvement across all error metrics, demonstrating that higher-dimensional representations enable more nuanced parameter estimation, albeit with increased computational cost. Disabling the linear tree option (A2) leads to a modest degradation in accuracy, while doubling the MLP hidden dimension (A3) slightly improves over the base model. The removal of the random projection layer (A4) degrades accuracy, confirming that this layer effectively diversifies low-dimensional tree embeddings into more expressive representations for the MLP decoder. Reducing the lag structure (A5) results in a notable drop in accuracy, emphasizing that an extended context is crucial for capturing the underlying temporal dependencies. Removing time series derived features (A6) hurts performance, highlighting an important property of the Hyper-Tree framework: since parameter learning is based on GBDTs, Hyper-Trees inherit the feature-based partitioning behavior characteristic of tree-models. This becomes critical in operational forecasting, where global models often struggle with adapting to the dynamics of individual series ([Godaheewa et al., 2021a](#)). This limitation, confirmed by

Table 4: Rossmann Store Sales Ablation Study Results.

Model	Ablation	MAPE	sMAPE	WAPE	RMSE	MAE	MASE
Hyper-Tree-AR	Base	8.830	9.023	8.881	798.023	622.134	0.358
	A2	8.638	8.811	8.692	780.799	608.668	0.351
	A5	9.069	8.936	8.996	822.958	623.887	0.358
	A6	10.343	10.428	10.251	921.092	708.853	0.415
	A11	24.753	21.315	21.423	1,791.4	1,478.2	0.844
Hyper-TreeNet-AR	Base	9.437	9.597	9.449	857.004	664.496	0.385
	A1	9.071	9.163	9.099	827.530	637.683	0.367
	A2	9.689	9.870	9.691	876.076	681.289	0.397
	A3	9.370	9.530	9.359	845.485	657.503	0.381
	A4	9.954	9.930	9.726	876.308	682.243	0.394
	A5	10.067	10.502	10.190	914.730	718.379	0.417
	A6	11.395	11.407	10.981	981.758	767.239	0.451
	A7	12.053	11.790	11.598	1,014.9	803.500	0.471
	A8	19.024	19.445	18.916	1,588.9	1,371.7	0.776
	A9	11.473	11.709	11.420	1,005.5	787.651	0.466
	A10	9.507	9.597	9.502	864.105	667.223	0.386
	A11	21.945	20.337	20.206	1,726.6	1,398.7	0.800

Reported are the mean forecast errors across series, with lower values indicating better performance and the best metrics highlighted in bold. Mean Absolute Percentage Error (MAPE); Symmetric Mean Absolute Percentage Error (sMAPE); Weighted Absolute Percentage Error (WAPE); Root Mean Squared Error (RMSE); Mean Absolute Error (MAE); Mean Absolute Scaled Error (MASE), which is calculated relative to the AutoETS forecasts (MAE of model / MAE of AutoETS).

observing similar degradation for the **LightGBM-AR** model under the same ablation, is not unique to Hyper-Trees but reflects a general characteristic of tree-based models when contextual features are missing.⁹ The drop in accuracy when GBDTs are excluded as the encoding layer and an MLP is used only (A7) demonstrates the critical role of tree-based feature encoding and validates the effectiveness of our GBDT-neural hybrid architecture. The substantial drop in accuracy when the **Hyper**-part is removed and the **TreeNet** is used to forecast directly (A8) validates that learning the parameters of a target model is crucial for the **Hyper-TreeNet**'s effectiveness. The two-stage approach (A9) shows markedly lower accuracy than our jointly trained model. This gap in accuracy likely stems from differences in optimization objective and representation quality. In the two-stage approach, the GBDT is optimized for direct forecasting rather than representation learning, creating outputs in the form of leaf indices that are not suited as inputs to the MLP for estimating parameters of the target model. In contrast, our end-to-end framework explicitly optimizes the GBDT for generating informative embeddings specifically relevant for estimating the target model parameters. This joint learning enables co-adaptation between components, with the network guiding the GBDT toward more useful representations than would be possible via independent training. While we opted for *Option 2* as the primary gradient method, results from A10 indicate negligible differences in model accuracy between the two gradient flow approaches for this particular forecasting task and dataset. Nevertheless, different problem domains or datasets might reveal more pronounced differences, and the impact of the gradient flow choice presents an interesting direction for future research. The most severe deterioration in accuracy is observed for A11, where time-varying parameters are replaced with constant AR-parameters averaged across the forecast

⁹The ablation metrics for **LightGBM-AR** are MAPE: 10.01; sMAPE: 9.68; WAPE: 9.62; RMSE: 855.75; MAE: 665.07.

horizon. This decline in accuracy provides evidence that the time-varying nature of parameters is an important component of our framework’s effectiveness. By constraining the model to constant parameters, we eliminate its ability to adapt to evolving patterns across the forecast horizon.

Turning to the **Hyper-Tree-AR** model, we observe that in contrast to the **Hyper-TreeNet-AR** model, disabling linear trees (A2) actually improves performance. Apart from this, the **Hyper-Tree-AR** shows similar sensitivity to a reduced lag structure (A5) and to removing the additional set of series-derived features (A6). Replacing time-varying parameters (A11) with constant parameters leads to the most substantial decline in accuracy of the **Hyper-Tree-AR** model, with MAPE nearly tripling and other metrics more than doubling compared to the base model. This result supports our conclusion that modeling parameters as functions of features is a critical aspect of our framework’s forecasting capability.

5. Framework Analysis and Considerations

The experiments presented in previous sections demonstrate the effectiveness of our Hyper-Tree framework across a range of forecasting tasks. In this section, we reflect on key architectural and conceptual aspects of Hyper-Trees, touching upon both their strengths and limitations. We focus on the following aspects: the conceptual differences between operating in parameter space versus function space, the inherent feature dependency characteristic of tree-based models and their implications for Hyper-Trees, the role of target model selection, the extrapolation capabilities that differentiate our approach from conventional tree-based forecasting models, and the constraints on parameter variation. These considerations help contextualize our empirical results and highlight practical implications for applying Hyper-Trees in real-world forecasting scenarios.

5.1. Parameter vs. Function Space

An important conceptual aspect of the Hyper-Tree framework is its operation in parameter space rather than function space, which introduces structural and interpretive advantages. In function space, each forecasted value represents a future time point, with potentially unbounded values ranging from negative to positive infinity. In contrast, the parameter space provides a more constrained and semantically meaningful setting. Parameters of classical time series models, such as ARIMA (Box et al., 2015) or Exponential Smoothing (Holt, 1957; Winters, 1960), correspond to specific temporal behaviors and carry inherent structural constraints. For instance, AR coefficients typically decay with increasing lag order, while smoothing parameters are bounded between 0 and 1. This structure aligns with known time series dynamics and introduces an inductive bias that likely supports more stable learning. A further advantage of operating in parameter space is the ability of Hyper-Trees to generate dynamic, time-varying parameters. Unlike traditional approaches where parameters remain fixed throughout the forecasting process, modeling parameters as functions of input features allows the framework to better adapt to evolving temporal patterns (Lee et al., 2023). The parameter-centric modeling design also enhances interpretability and facilitates the incorporation of domain knowledge. Parameters with clear theoretical or practical meaning allow for easier validation, and domain-specific constraints can be readily encoded into the parameters, improving the model’s ability to create plausible forecasts. Operating in parameter space also extends to the embeddings in the Hyper-TreeNet architecture. Rather than learning representations optimized for direct forecasting, the tree component generates embeddings specifically

tailored for parameter estimation. These embeddings capture feature relationships most relevant for estimating time-varying coefficients, which the neural network then maps to the actual target model parameters. This ensures that both the tree embeddings and the network outputs operate within the same parameter space, maintaining consistency throughout the modeling process and facilitating effective learning of time-varying parameters.

5.2. Feature Dependency

Since parameter learning is based on GBDTs, Hyper-Trees inherit the characteristic feature-based partitioning of tree-based models. Unlike neural networks that can learn hierarchical representations from time series data through their multi-layer architectures, tree-based models require explicit features to capture relevant patterns. As with conventional GBDTs, the quality of features directly influences the Hyper-Tree’s ability to learn meaningful parameter mappings. Previous research has shown that global models can benefit from operating on more homogeneous subsets of data (Godaheva et al., 2021a), suggesting that effective feature-based partitioning can improve model performance. This is particularly important since global forecasting models often struggle with not being localized enough to particular series, especially in datasets with heterogeneous series. As such, any global tree-based model, including our Hyper-Tree framework, requires features that effectively identify similar patterns and create homogeneous groups. In operational contexts with rich feature sets, our framework demonstrates competitive accuracy. However, in scenarios with limited feature availability or where relevant information are not adequately captured in the features, any tree-based model faces limitations. This characteristic differentiates Hyper-Trees from deep learning approaches like Deep-AR (Salinas et al., 2020) or TFT (Lim et al., 2021), which learn complex temporal representations through their recurrent or attention-based architectures. While our experiments show that Hyper-Trees perform well across various benchmarks with relatively simple feature sets, practitioners should be mindful of this requirement when applying our framework.

5.3. Target Model Dependency

A defining characteristic of the Hyper-Tree framework is its reliance on a pre-specified target model, whose parameters are learned as functions of input features. This design choice introduces a strong inductive bias that can be both an advantage and a limitation. The accuracy of Hyper-Trees is intrinsically bound to how well the selected target model aligns with the underlying dynamics of the data. Our experiments demonstrate that autoregressive (AR) (Box et al., 2015) and exponential smoothing (ETS) (Holt, 1957; Winters, 1960) specifications perform well across a range of datasets. However, both impose structural assumptions that may not hold. For instance, AR models assume specific relationships between lagged observations, while ETS models impose specific assumptions on trend and seasonality components. When time series characteristics deviate from these assumptions, the limitation stems not from the Hyper-Tree framework itself, but from the choice of the target model. An inappropriate specification can impair forecast accuracy, underscoring the importance of careful model selection. As with any modeling approach, practitioners should select target models that reflect the underlying data-generating process. One might argue that models like Deep-AR (Salinas et al., 2020), TFT (Lim et al., 2021) or LightGBM (Ke et al., 2017) are more flexible when applied to highly irregular operational time series, as they can learn arbitrary functions without imposing strict structural assumptions on the data-generating process.

This may hold true in certain settings and datasets, as these models can capture a broad range of dependencies without being constrained to the inductive bias of a predefined target model. In principle, a sufficiently complex neural network or tree ensemble can approximate any data-generating process given enough data and model capacity. However, this generality may come at a cost. Without the guidance of a well-justified inductive bias that can guide forecasts toward temporal structures, these models may require substantially more data to learn patterns or may not explicitly account for structural temporal relationships that are well-understood in time series analysis. Hyper-Trees, by contrast, balance flexibility with structure, leveraging the interpretability and structural assumptions embedded in classical time series models with the flexible feature learning capabilities of tree-based approaches.

The dependency on a suitably chosen target model becomes evident when forecasting specific types of time series. For example, intermittent time series, characterized by sporadic patterns with frequent zero values, pose significant forecasting challenges for conventional AR or ETS models. This is particularly relevant for datasets like the M5 data (Makridakis et al., 2022a,b), where the most disaggregated series exhibit highly intermittent patterns. Such series require specialized approaches that can accommodate both the frequency of zero values and the magnitude of non-zero occurrences. The modularity of our architecture, however, can be extended to incorporate more specialized target models. Future research could explore integrating models specifically designed for intermittent demand, e.g., Croston’s method (Croston, 1972). When such specialized models replace standard AR or ETS target models, the Hyper-Tree framework learns the relevant parameters, e.g., zero probability and demand size, as functions of features.

5.4. *Extrapolation Properties*

The inability of tree-based models to forecast values beyond the range of the training data is an important limitation to consider when evaluating their practical utility. This raises the natural question about the extrapolation properties of the Hyper-Tree framework and touches upon a fundamental characteristic of tree-based models, i.e., their piece-wise constant nature. This piece-wise constant constraint intersects with two main components of the Hyper-Tree architecture: the generation of model parameters and the creation of tree-embeddings. We first explore how our approach addresses the piece-wise constant limitations in parameter generation, followed by a discussion of tree-embeddings.

Parameter Generation. The Hyper-TreeNet variant mitigates this limitation through its network-based parameter generation. Unlike conventional tree-based models that create piece-wise constant outputs, the Hyper-TreeNet uses an MLP to generate the target model parameters. The tree-based component creates embeddings first which are then transformed by the MLP into target model parameters. Beyond the Hyper-TreeNet variant, Hyper-Tree models can benefit from the linear-tree extension introduced by Shi et al. (2019), which replaces piece-wise constant with piece-wise linear trees. Importantly, our ablation studies indicate that replacing linear with piece-wise constant trees has only marginal effects on forecast accuracy for both **Hyper-Tree-AR** and **Hyper-TreeNet-AR** models, with accuracy even slightly improving when using piece-wise constant parameter outputs for the former. This highlights a crucial distinction between our Hyper-Tree architecture and conventional GBDTs. Even when Hyper-Tree parameters are subject to piece-wise constant restrictions, the resulting forecasts can effectively extend beyond the training data range.

This capability results from our approach’s design principle of operating in parameter space, where parameters are first generated and then applied to the target model, preventing the piece-wise constant constraint from transferring to the forecasts themselves. Additionally, features at each time step translate into context-specific parameters, allowing the model to adapt to changing conditions even if the parameter values are constrained to values within the range learned during training.

Embedding Generation. While our Hyper-Tree framework addresses the extrapolation limitations of traditional GBDTs, it is important to acknowledge that the tree-based encoding component without the linear-tree option of Shi et al. (2019) is subject to piece-wise constant constraints. In the Hyper-TreeNet architecture, the GBDT component partitions the feature space through hierarchical binary splits, creating regions where all observations within the same leaf node receive identical embedding values. This discretization limits tree embeddings to a finite number of distinct values determined by the number of leaf nodes across all trees. However, these discrete embeddings only serve as intermediate representations that are subsequently transformed by the MLP component, which effectively generates continuous parameter values. Additionally, the random projection matrix helps diversify the representations before MLP processing, further mitigating limitations of the piece-wise constant tree embeddings. Our ablation studies indicate that removing the linear-tree option (Shi et al., 2019) has only minor effects on embedding quality and overall forecast accuracy. Nevertheless, a more comprehensive analysis of how tree-generated embeddings impact forecast accuracy presents an important direction for future research.

5.5. On Parameter Variation

A potential concern with our time-varying parameter framework might be that, if coefficients were allowed to vary unconstrained over time, the model class could become arbitrarily flexible. Such a model could fit any training series perfectly, but at the cost of generalization and interpretability. Our Hyper-Tree framework mitigates this concern because parameters are functions of features, not unconstrained values that vary freely over time. More formally, Hyper-Trees model parameters as $\theta_{j,it} = f_j(\mathbf{x}_{it})$, where $\mathbf{x}_{it}$ represents features, i.e., temporal indicators, as well as categorical and numeric covariates. This imposes a consistency constraint in the sense that similar feature contexts yield similar parameters. For instance, in the **Air Passengers** dataset with $\mathbf{x}_{it} = \{\text{month}_t, \text{quarter}_t\}$ being the only features, all January observations across different years share identical parameters, as illustrated in Figure F1. This parameter consistency holds because all temporal features used across the experiments are date-derived (quarter, month, week, day-of-week, etc.) rather than raw time indices ($t = 1, \dots, T$), except for the STL-decomposition example in Section 4.1. This ensures that parameter variation is tied to temporal patterns rather than to the passage of time itself. Further, the learned mapping $f(\cdot)$ is capacity-constrained through gradient boosting regularization, including tree depth and constraints on leaf size and leaf values. This keeps parameter variation tied to meaningful feature patterns rather than allowing for unconstrained coefficient paths. The same mechanism also preserves parameter interpretability. In contrast to an unconstrained specification with coefficients free to vary at each time step, where many parameter paths can yield indistinguishable forecasts, our feature-driven approach anchors parameters to observable covariates through the learned mapping $f(\cdot)$. Practitioners can examine how coefficients respond to specific conditions such as promotions, holidays, or store characteristics.¹⁰

¹⁰See Figures F1, F2 and F3 for examples.

Beyond these architectural considerations, our experimental results provide evidence that the time-varying parameters do not collapse into unconstrained flexibility. If Hyper-Trees were effectively learning arbitrary time-varying coefficient paths, one would expect poor out-of-sample accuracy. Instead, Hyper-Tree models consistently show strong accuracy across datasets and often outperform their static-parameter counterparts (Tables 1, 2 and 3). This pattern indicates that the learned parameters capture genuine temporal structure rather than overfitting to training patterns.

6. Conclusion and Future Research

With the Hyper-Tree framework, we have ventured into the comparatively unexplored use of gradient boosted trees for modeling the parameters of classical time series models. By repurposing GBDTs to learn the parameters of target models, we have combined the effectiveness of boosted trees on tabular data with the strengths of classical time series models. This approach naturally induces a time series inductive bias to tree models and addresses the limitations of conventional GBDT-based forecasts. We also suggest a novel approach to resolve the scaling issue of GBDTs, contributing to the growing literature on combining decision trees and neural networks. In this hybrid approach, the tree serves as an encoding layer, transforming features into informative representations, while the network functions as a decoder, mapping these embeddings to the target model parameters. Through comprehensive experiments, we demonstrate the effectiveness of our framework across several forecasting scenarios and datasets. Our findings demonstrate the potential of tree-based models for learning the parameters of target models, bridging the gap between traditional time series methods and modern gradient boosting approaches. This perspective opens new ways of understanding and using boosted tree models beyond their conventional applications in time series forecasting.

Looking forward, the horizon of future research in this domain is broad and inviting. While we have introduced an initial conception of tree-based hyper-models, there remain several compelling avenues for future investigation. Our current focus on relatively simple target models opens up opportunities to extend our approach to more complex architectures. Exploring more general state space formulations would offer a unified and flexible framework for modeling complex temporal dynamics, while more specialized target models tailored to specific time series characteristics, such as intermittent demand, pose another valuable extension. Extending the autoregressive target model to vector-autoregressive (VAR) formulations (Sims, 1980; Lütkepohl, 2005) would enable the framework to capture cross-series dependencies, moving from univariate to multivariate forecasting. Also, extending the framework to include automated model and lag selection, along with sparsity-inducing penalties on target model parameters, would further enhance the adaptability of our Hyper-Tree models. Additionally, improving the runtime efficiency of the **Hyper-Tree-ETS** would broaden its applicability to larger-scale problems, making it a more versatile tool for diverse time series forecasting applications. A probabilistic extension of our approach also merits consideration. Incorporating uncertainty quantification into the Hyper-Tree framework would offer a more comprehensive understanding of the underlying time series dynamics. Adapting statistical inference frameworks for time-varying parameters to our Hyper-Tree context would be another valuable extension, enabling the construction of confidence intervals, hypothesis testing, and formal assessment of tree-based time-varying parameters. Besides extending the approach of Spuck et al. (2025) to a time series context, Bayesian tree ensembles such as BART (Chipman et al.,

2010) offer another potential direction for such inference, as replacing the GBDT component that learns the parameter mapping with a Bayesian formulation would provide posterior distributions over the target model parameters directly. The impact of gradient flow options on model behavior across different domains and datasets presents another interesting area for future exploration. Our ablation studies show minimal differences between shared and separated gradient approaches, but this may vary in other contexts. A more comprehensive analysis of tree-based embeddings and their representational capacity poses another interesting direction for future research. Beyond these extensions, combining gradient boosted trees and neural networks leverages the complementary strengths of both model classes, bridging tree-based and deep learning approaches. This integration points toward a class of hybrid models that open new avenues for flexible forecasting architectures.

Implementation

The code will be made available on the official repository <https://github.com/StatMixedML/Hyper-Trees> upon the final publication of this paper.

References

- Alexandrov, A., Benidis, K., Bohlke-Schneider, M., Flunkert, V., Gasthaus, J., Januschowski, T., Maddix, D. C., Rangapuram, S., Salinas, D., Schulz, J., Stella, L., Türkmen, A. C., & Wang, Y. (2020). GluonTS: Probabilistic and Neural Time Series Modeling in Python. *Journal of Machine Learning Research*, 21, 1–6.
- Ansari, A. F., Stella, L., Turkmen, C., Zhang, X., Mercado, P., Shen, H., Shchur, O., Rangapuram, S. S., Arango, S. P., Kapoor, S., Zschiegner, J., Maddix, D. C., Wang, H., Mahoney, M. W., Torkkola, K., Wilson, A. G., Bohlke-Schneider, M., & Wang, Y. (2024). Chronos: Learning the Language of Time Series. *Published in Transactions on Machine Learning Research (10/2024)*, .
- Athanasopoulos, G., Hyndman, R. J., Song, H., & Wu, D. C. (2011). The tourism forecasting competition. *International Journal of Forecasting*, 27, 822–844.
- Badirli, S., Liu, X., Xing, Z., Bhowmik, A., & Keerthi, S. S. (2020). Gradient Boosting Neural Networks: GrowNet. *arXiv Pre-Print, abs/2002.07971*.
- Bansal, M. A., Sharma, R., & Kathuria, M. (2022). A Systematic Review on Data Scarcity Problem in Deep Learning: Solution and Applications. *ACM Comput. Surv.*, 54.
- Belsley, D. A. (1973a). A Test for Systematic Variation in Regression Coefficients. *Annals of Economic and Social Measurement*, 2, 495–499.
- Belsley, D. A. (1973b). On the Determination of Systematic Parameter Variation in the Linear Regression Model. *Annals of Economic and Social Measurement*, 2, 487–494.
- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5, 157–166.
- Berger, M., Tutz, G., & Schmid, M. (2019). Tree-Structured Modelling of Varying Coefficients. *Statistics and Computing*, 29, 217–229.
- Bollerslev, T. (1986). Generalized autoregressive conditional heteroskedasticity. *Journal of Econometrics*, 31, 307–327.
- Box, G. E. P., Jenkins, G. M., Reinsel, G. C., & Ljung, G. M. (2015). *Time Series Analysis: Forecasting and Control*. Wiley Series in Probability and Statistics (5th ed.). Wiley.
- Bürgin, R., & Ritschard, G. (2017). Coefficient-Wise Tree-Based Varying Coefficient Regression with vcrpart. *Journal of Statistical Software*, 80, 1–33.
- Cai, Z., Fan, J., & Yao, Q. (2000). Functional-Coefficient Regression Models for Nonlinear Time Series. *Journal of the American Statistical Association*, 95, 941–956.
- Casale, P., Pujol, O., & Radeva, P. (2011). Embedding Random Projections in Regularized Gradient Boosting Machines. In O. Okun, G. Valentini, & M. Re (Eds.), *Ensembles in Machine Learning Applications* (pp. 201–216). Berlin, Heidelberg: Springer Berlin Heidelberg.

- Chen, R., & Tsay, R. S. (1993). Functional-Coefficient Autoregressive Models. *Journal of the American Statistical Association*, 88, 298–308.
- Chen, S.-A., Li, C.-L., Arik, S. Ö., Yoder, N. C., & Pfister, T. (2023). TSMixer: An All-MLP Architecture for Time Series Forecasting. *Transactions on Machine Learning Research*, .
- Chen, T., & Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* KDD '16 (pp. 785–794).
- Chipman, H. A., George, E. I., & McCulloch, R. E. (2010). BART: Bayesian Additive Regression Trees. *The Annals of Applied Statistics*, 4, 266–298.
- Cleveland, R. B., Cleveland, W. S., McRae, J. E., & Terpenning, I. J. (1990). STL: A seasonal-trend decomposition procedure based on loess. *Journal of Official Statistics*, 6, 3–33.
- Cooley, T. F., & Prescott, E. C. (1973). Systematic (Non-Random) Variation Models Varying Parameter Regression: A Theory And Some Applications. *Annals of Economic and Social Measurement*, 2, 463–473.
- Cooley, T. F., & Prescott, E. C. (1976). Estimation in the Presence of Stochastic Parameter Variation. *Econometrica*, 44, 167–184.
- Corporación Favorita, Elliott, J., & McDonald, M. (2017). Corporación Favorita Grocery Sales Forecasting. URL: <https://kaggle.com/competitions/favorita-grocery-sales-forecasting>.
- Croston, J. D. (1972). Forecasting and Stock Control for Intermittent Demands. *Journal of the Operational Research Society*, 23, 289–303.
- Dahlhaus, R. (2000). A Likelihood Approximation for Locally Stationary Processes. *The Annals of Statistics*, 28, 1762–1794.
- Dahlhaus, R. (2012). 13 - Locally Stationary Processes. In Tata Subba Rao, Suhasini Subba Rao, & C.R. Rao (Eds.), *Time Series Analysis: Methods and Applications* (pp. 351–413). Elsevier volume 30 of *Handbook of Statistics*.
- Das, A., Kong, W., Sen, R., & Zhou, Y. (2024). A decoder-only foundation model for time-series forecasting. In *Proceedings of the 41st International Conference on Machine Learning ICML'24*. JMLR.org.
- Delong, L., Lindholm, M., & Zakrisson, H. (2023). On Cyclic Gradient Boosting Machines. SSRN working paper 4352505. Available at SSRN.
- Duan, T., Anand, A., Ding, D. Y., Thai, K. K., Basu, S., Ng, A. Y., & Schuler, A. (2020). NGBoost: Natural Gradient Boosting for Probabilistic Prediction. In *Proceedings of the 37th International Conference on Machine Learning, ICML 2020, 13-18 July 2020, Virtual Event* (pp. 2690–2700). PMLR volume 119 of *Proceedings of Machine Learning Research*.

- Duan, W., He, X., Zhou, L., Thiele, L., & Rao, H. (2022). Combating Distribution Shift for Accurate Time Series Forecasting via Hypernetworks. In *2022 IEEE 28th International Conference on Parallel and Distributed Systems (ICPADS)* (pp. 900–907). Los Alamitos, CA, USA: IEEE Computer Society.
- Ekambaram, V., Jati, A., Dayama, P., Mukherjee, S., Nguyen, N. H., Gifford, W. M., Reddy, C., & Kalagnanam, J. (2024). Tiny Time Mixers (TTMs): Fast Pre-trained Models for Enhanced Zero/Few-Shot Forecasting of Multivariate Time Series. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*.
- Emami, S., & Martínez-Muñoz, G. (2022). Multioutput Regression Neural Network Training via Gradient Boosting. *ESANN 2022 proceedings*, .
- Engle, R. F. (1982). Autoregressive Conditional Heteroskedasticity with Estimates of the Variance of United Kingdom Inflation. *Econometrica*, 50, 987–1007.
- Fons, E., Sztrajman, A., El-Laham, Y., Iosifidis, A., & Vyetrenko, S. (2022). HyperTime: Implicit Neural Representations for Time Series. In *NeurIPS 2022 Workshop on Synthetic Data for Empowering ML Research*.
- Friedman, J. H. (2001). Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29, 1189–1232.
- Garza, F., Mergenthaler Canseco, M., Challú, C., & Olivares, K. G. (2022). StatsForecast: Lightning fast forecasting with statistical and econometric models. *PyCon Salt Lake City, Utah, US 2022*, <https://github.com/Nixtla/statsforecast>, .
- Godahehwa, R., Bandara, K., Webb, G. I., Smyl, S., & Bergmeir, C. (2021a). Ensembles of localised models for time series forecasting. *Knowledge-Based Systems*, 233, 107518.
- Godahehwa, R., Bergmeir, C., Webb, G. I., Hyndman, R. J., & Montero-Manso, P. (2021b). Monash Time Series Forecasting Archive. In *Neural Information Processing Systems Track on Datasets and Benchmarks*.
- Godahehwa, R., Webb, G. I., Schmidt, D., & Bergmeir, C. (2023). SETAR-Tree: a novel and accurate tree algorithm for global time series forecasting. *Machine Learning*, 112, 2555–2591.
- Goswami, M., Szafer, K., Choudhry, A., Cai, Y., Li, S., & Dubrawski, A. (2024). MOMENT: a family of open time-series foundation models. In *Proceedings of the 41st International Conference on Machine Learning ICML’24*. JMLR.org.
- Ha, D., Dai, A. M., & Le, Q. V. (2017). HyperNetworks. In *International Conference on Learning Representations*.
- Hasson, H., Wang, B., Januschowski, T., & Gasthaus, J. (2021). Probabilistic Forecasting: A Level-Set Approach. In A. Beygelzimer, Y. Dauphin, P. Liang, & J. Wortman Vaughan (Eds.), *Advances in Neural Information Processing Systems*.

- Hastie, T., & Tibshirani, R. (1993). Varying-Coefficient Models. *Journal of the Royal Statistical Society: Series B (Methodological)*, 55, 757–796.
- He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., & Candela, J. Q. (2014). Practical Lessons from Predicting Clicks on Ads at Facebook. In *Proceedings of the Eighth International Workshop on Data Mining for Online Advertising ADKDD'14* (pp. 1–9). New York, NY, USA: Association for Computing Machinery.
- Holt, C. C. (1957). Forecasting seasonals and trends by exponentially weighted moving averages. (*ONR Memorandum No. 52*). *Carnegie Institute of Technology, Pittsburgh USA*. Reprinted in the *International Journal of Forecasting* 2004, 20, 5–10.
- Horrell, M. (2025). GBNet: Gradient Boosting packages integrated into PyTorch. *Journal of Open Source Software*, 10, 8047.
- Hutchinson, M. (1990). A stochastic estimator of the trace of the influence matrix for laplacian smoothing splines. *Communications in Statistics - Simulation and Computation*, 19, 433–450.
- Hyndman, R., Kang, Y., Montero-Manso, P., O'Hara-Wild, M., Talagala, T., Wang, E., & Yang, Y. (2024). tsfeatures: Time Series Feature Extraction. URL: <https://github.com/Nixtla/tsfeatures>.
- Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice*. (3rd ed.). Australia: OTexts.
- Hyndman, R. J., & Koehler, A. B. (2006). Another look at measures of forecast accuracy. *International Journal of Forecasting*, 22, 679–688.
- Iosipoi, L., & Vakhrushev, A. (2022). SketchBoost: fast gradient boosted decision tree for multioutput problems. In *Proceedings of the 36th International Conference on Neural Information Processing Systems NIPS '22*. Red Hook, NY, USA: Curran Associates Inc.
- Januschowski, T., Gasthaus, J., Wang, Y., Salinas, D., Flunkert, V., Bohlke-Schneider, M., & Calot, L. (2020). Criteria for classifying forecasting methods. *International Journal of Forecasting*, 36, 167–177.
- Januschowski, T., & Kolassa, S. (2019). A Classification of Business Forecasting Problems. *Forecast: The International Journal of Applied Forecasting*, 1, 36–43.
- Januschowski, T., Wang, Y., Torkkola, K., Erkkilä, T., Hasson, H., & Gasthaus, J. (2022). Forecasting with trees. *International Journal of Forecasting*, 38, 1473–1481.
- Johansson, U., Löfström, T., & Boström, H. (2023). Conformal Predictive Distribution Trees. *Annals of Mathematics and Artificial Intelligence*, .
- Karingula, S. R., Ramanan, N., Tahmasbi, R., Amjadi, M., Jung, D., Si, R., Thimmisetty, C., Polania, L. F., Sayer, M., Taylor, J., & Coelho, C. N. (2022). Boosted Embeddings for Time-Series Forecasting. In G. Nicosia, V. Ojha, E. La Malfa, G. La Malfa, G. Jansen, P. M. Pardalos, G. Giuffrida, & R. Umeton (Eds.), *Machine Learning, Optimization, and Data Science* (pp. 1–14). Cham: Springer International Publishing.

- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Proceedings of the 31st International Conference on Neural Information Processing Systems NIPS’17* (pp. 3149–3157).
- Király, F., Löning, M., Bagnall, T., Middlehurst, M., Ganesh, S., Walter, M., Oastler, G., Lines, J., Ray, A., Kaz, V., Mentel, L., Heidrich, B., Mishra, S., holder, C., Bartling, D., Tsaprounis, L., Kuhns, R. N., Gilbert, C., Baichoo, M., Akmal, H., Guzal, Owoseni, T., Rockenschaub, P., eenticott shell, Alavi, S., jesellier, Manani, K., Schäfer, P., & Khrapov, S. (2024). sktime/sktime: v0.29.0.
- Knauer, F., & Cukierski, W. (2015). Rossmann Store Sales. URL: <https://kaggle.com/competitions/rossmann-store-sales>.
- Kontschieder, P., Fiterau, M., Criminisi, A., & Bulò, S. R. (2016). Deep Neural Decision Forests. In *Proceedings of the Twenty-Fifth International Joint Conference on Artificial Intelligence IJ-CAI’16* (pp. 4190–4194). AAAI Press.
- Lee, J., Kim, C., Lee, G., Lim, H., Choi, J., Lee, K., Lee, D., Hong, S., & Park, N. (2023). HyperNetwork Approximating Future Parameters for Time Series Forecasting under Temporal Drifts. In *Advances in Neural Information Processing Systems Workshop on Distribution Shifts (NeurIPS DistShift) NIPS’2023* (pp. 1–17).
- Li, H., Song, J., Xue, M., Zhang, H., Ye, J., Cheng, L., & Song, M. (2023). A Survey of Neural Trees. *ACM Computing Survey*, 1.
- Lim, B., Arık, S. Ö., Loeff, N., & Pfister, T. (2021). Temporal Fusion Transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37, 1748–1764.
- Löning, M., Bagnall, A., Ganesh, S., Kazakov, V., Lines, J., & Király, F. J. (2019). sktime: A Unified Interface for Machine Learning with Time Series. *Workshop on Systems for ML at NeurIPS 2019*, .
- Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., Katz, R., Himmelfarb, J., Bansal, N., & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2, 2522–5839.
- Lundberg, S. M., & Lee, S.-I. (2017). A Unified Approach to Interpreting Model Predictions. In I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems 30* (pp. 4765–4774). Curran Associates, Inc.
- Lütkepohl, H. (2005). *New Introduction to Multiple Time Series Analysis*. Springer.
- Makridakis, S., & Hibon, M. (2000). The M3-Competition: results, conclusions and implications. *International Journal of Forecasting*, 16, 451–476.
- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2018). The M4 Competition: Results, findings, conclusion and way forward. *International Journal of Forecasting*, 34, 802–808.

- Makridakis, S., Spiliotis, E., & Assimakopoulos, V. (2022a). The M5 competition: Background, organization, and implementation. *International Journal of Forecasting*, 38, 1325–1336.
- Makridakis, S., Spiliotis, E., Assimakopoulos, V., Chen, Z., Gaba, A., Tsetlin, I., & Winkler, R. L. (2022b). The M5 uncertainty competition: Results, findings and conclusions. *International Journal of Forecasting*, 38, 1365–1385.
- Martens, J. (2020). New Insights and Perspectives on the Natural Gradient Method. *Journal of Machine Learning Research*, 21, 1–76.
- März, A. (2019). XGBoostLSS - An extension of XGBoost to probabilistic forecasting. *arXiv Pre-Print*, (pp. 1–23).
- März, A. (2022). Multi-Target XGBoostLSS Regression. *arXiv Pre-Print*, (pp. 1–11).
- März, A., & Kneib, T. (2022). Distributional Gradient Boosting Machines. *arXiv Pre-Print*, (pp. 1–15).
- Medeiros, M. C., & Veiga, A. (2000). A hybrid linear-neural model for time series forecasting. *IEEE Transactions on Neural Networks*, 11, 1402–1412.
- Montero-Manso, P., & Hyndman, R. J. (2021). Principles and algorithms for forecasting groups of time series: Locality and globality. *International Journal of Forecasting*, 37, 1632–1653.
- Nasios, I., & Vogklis, K. (2022). Blending gradient boosted trees and neural networks for point and probabilistic forecasting of hierarchical time series. *International Journal of Forecasting*, 38, 1448–1459.
- Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization*. Springer Series in Operations Research and Financial Engineering (2nd ed.). New York, NY: Springer.
- O’Hara-Wild, M., Hyndman, R., Wang, E., & Godahewa, R. (2022). tsibbledata: Diverse Datasets for ‘tsibble’.
- Pascanu, R., Mikolov, T., & Bengio, Y. (2013). On the difficulty of training recurrent neural networks. In *Proceedings of the 30th International Conference on International Conference on Machine Learning - Volume 28 ICML’13* (p. III–1310–III–1318). JMLR.org.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Curran Associates Inc volume 721.
- Popov, S., Morozov, S., & Babenko, A. (2020). Neural Oblivious Decision Ensembles for Deep Learning on Tabular Data. *8th International Conference on Learning Representations*, .
- Primiceri, G. E. (2005). Time Varying Structural Vector Autoregressions and Monetary Policy. *The Review of Economic Studies*, 72, 821–852.

- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems*. Curran Associates, Inc volume 31.
- Puth, M.-T., Tutz, G., Heim, N., Münster, E., Schmid, M., & Berger, M. (2020). Tree-Based Modeling of Time-Varying Coefficients in Discrete Time-to-Event Models. *Lifetime Data Analysis*, 26, 545–572.
- Rangapuram, S. S., Seeger, M. W., Gasthaus, J., Stella, L., Wang, Y., & Januschowski, T. (2018). Deep State Space Models for Time Series Forecasting. In S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, & R. Garnett (Eds.), *Advances in Neural Information Processing Systems*. Curran Associates, Inc volume 31.
- Rasul, K., Ashok, A., Williams, A. R., Ghonia, H., Bhagwatkar, R., Khorasani, A., Bayazi, M. J. D., Adamopoulos, G., Riachi, R., Hassen, N., Biloš, M., Garg, S., Schneider, A., Chapados, N., Drouin, A., Zantedeschi, V., Nevmyvaka, Y., & Rish, I. (2023). Lag-Llama: Towards Foundation Models for Time Series Forecasting. In *R0-FoMo: Workshop on Robustness of Few-shot and Zero-shot Learning in Foundation Models at NeurIPS 2023*.
- Romano, Y., Patterson, E., & Candès, E. J. (2019). Conformalized quantile regression. In *Proceedings of the 33rd International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc.
- Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1, 206–215.
- Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning Representations by Back-Propagating Errors. *Nature*, 323, 533–536.
- Salinas, D., Flunkert, V., Gasthaus, J., & Januschowski, T. (2020). DeepAR: Probabilistic forecasting with autoregressive recurrent networks. *International Journal of Forecasting*, 36, 1181–1191.
- Seabold, S., & Perktold, J. (2010). statsmodels: Econometric and statistical modeling with python. In *9th Python in Science Conference*.
- Shi, Y., Li, J., & Li, Z. (2019). Gradient Boosting with Piece-Wise Linear Regression Trees. In *Proceedings of the 28th International Joint Conference on Artificial Intelligence IJCAI’19* (pp. 3432–3438). AAAI Press.
- Sigrist, F. (2021). Gradient and Newton boosting for classification and regression. *Expert Systems with Applications*, 167, 114080.
- Sims, C. A. (1980). Macroeconomics and Reality. *Econometrica*, 48, 1–48. doi:[10.2307/1912017](https://doi.org/10.2307/1912017).
- Smyl, S. (2020). A hybrid method of exponential smoothing and recurrent neural networks for time series forecasting. *International Journal of Forecasting*, 36, 75–85.

- Sprangers, O., Schelter, S., & de Rijke, M. (2021). Probabilistic Gradient Boosting Machines for Large-Scale Probabilistic Regression. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining KDD '21* (pp. 1510–1520).
- Sprangers, O., Schelter, S., & de Rijke, M. (2023). Parameter-efficient deep probabilistic forecasting. *International Journal of Forecasting*, 39, 332–345.
- Spuck, N., Schmid, M., Monin, M., & Berger, M. (2025). Confidence Intervals for Tree-Structured Varying Coefficients. *Computational Statistics & Data Analysis*, 207, 108142.
- Vandeput, N. (2024). VN1 Forecasting - Accuracy Challenge. URL: <https://www.datasource.ai/en/home/data-science-competitions-for-startups/phase-2-vn1-forecasting-accuracy-challenge/description>.
- de Vito, L. (2017). LinXGBoost: Extension of XGBoost to Generalized Local Linear Models. *arXiv Pre-Print*, (pp. 1–12).
- Vogelsgesang, A., Haubenschild, M., Finis, J., Kemper, A., Leis, V., Muehlbauer, T., Neumann, T., & Then, M. (2018). Get Real: How Benchmarks Fail to Represent the Real World. In *Proceedings of the Workshop on Testing Database Systems DBTest'18*. New York, NY, USA: Association for Computing Machinery.
- Vovk, V., Gammerman, A., & Shafer, G. (2022). *Algorithmic Learning in a Random World*. (2nd ed.). Berlin, Heidelberg: Springer-Verlag.
- Wang, J. C., & Hastie, T. (2014). Boosted Varying-Coefficient Regression Models for Product Demand Prediction. *Journal of Computational and Graphical Statistics*, 23, 361–382.
- Winters, P. R. (1960). Forecasting Sales by Exponentially Weighted Moving Averages. *Management Science*, 6, 324–342.
- Woo, G., Liu, C., Kumar, A., Xiong, C., Savarese, S., & Sahoo, D. (2024). Unified training of universal time series forecasting transformers. In *Proceedings of the 41st International Conference on Machine Learning ICML'24*. JMLR.org.
- Wood, S. N. (2017). *Generalized Additive Models: An Introduction with R*. (2nd ed.). Boca Raton: Chapman and Hall/CRC.
- Yeh, C.-C. M., Fan, Y., Dai, X., Saini, U. S., Lai, V., Aboagye, P. O., Wang, J., Chen, H., Zheng, Y., Zhuang, Z., Wang, L., & Zhang, W. (2024). RPMixer: Shaking Up Time Series Forecasting with Random Projections for Large Spatial-Temporal Data. In *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining KDD '24* (pp. 3919–3930). New York, NY, USA: Association for Computing Machinery.
- Yousuf, K., & Ng, S. (2021). Boosting high dimensional predictive regressions with time varying parameters. *Journal of Econometrics*, 224, 60–87.
- Zakrisson, H., & Lindholm, M. (2025). A Tree-Based Varying Coefficient Model. *Computational Statistics*, 40, 5105–5134.

- Zeng, A., Chen, M., Zhang, L., & Xu, Q. (2023). Are transformers effective for time series forecasting? In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence AAAI'23/IAAI'23/EAAI'23*. AAAI Press.
- Zhou, Y., & Hooker, G. (2022). Decision Tree Boosted Varying Coefficient Models. *Data Mining and Knowledge Discovery*, 36, 2237–2271.

Declaration of Interest

The authors declare no competing interests.

Appendices

Appendix A. Notation

The following table summarizes the notation used throughout the paper.

Table A1: Paper Notation.

Symbol	Description
<i>Core components</i>	
$\mathcal{T}$	Hyper-Tree (GBDT component)
$\mathcal{M}$	Target time series model (e.g., AR(p), ETS)
$\mathcal{N}$	Neural network (MLP) in the Hyper-TreeNet architecture
$\mathcal{L}$	Loss function
$\mathcal{L}_2$	Mean squared error (MSE) loss
$\mathcal{L}_{\text{split}}$	Loss reduction from a candidate split
$\tau_{0.5}$	Quantile loss evaluated at the median
NLL_{T}	Negative log-likelihood of a Student-T distribution
<i>Data and indexing</i>	
We use joint panel indexing $\{it\}$ (no comma) when the subscript identifies one observation (series i at time t), including shifted time as in y_{it-j} . Commas separate a parameter label from the observation index, as in $\theta_{j,it}$. Local / single-series contexts drop the i subscript.	
y_i, x_i	Target and feature vector for the i -th observation (generic GBDT context), $i = 1, \dots, N$
$y_{it}, \mathbf{x}_{it}$	Target and feature vector for series i at time t (panel / global time series context)
$y_t, \mathbf{x}_t$	Target and feature vector at time t (local / single-series context)
X_{Tree}	Feature space of the Hyper-Tree
N	Number of observations, with index $i = 1, \dots, N$ (distinct from the calligraphic $\mathcal{N}$)
P	Number of target model parameters, with index $j = 1, \dots, P$
C	Dimension of the feature space
p	Number of AR lags
t	Time index, $t = 1, \dots, T$
T	Length of the time series (distinct from the calligraphic $\mathcal{T}$)
h	Forecast horizon
m	Boosting iteration, with $m = 1, \dots, M$
M	Total number of boosting iterations (distinct from the calligraphic $\mathcal{M}$)
$f : \mathcal{X}^C \rightarrow \Theta^P$	Learned global mapping from features to target model parameters
<i>Tree outputs and updates</i>	
$\hat{\psi}_{\mathcal{T}}$	Output of the GBDT (tree output)
$\hat{\psi}_{\mathcal{M}}$	Forecast of the target time series model
$\hat{\psi}_{i\mathcal{T}}^{(m)}$	Tree output for observation i at iteration m
$\hat{\delta}_{i\mathcal{T}}^{(m)}$	Incremental update at iteration m
ν	Learning rate
$q(\cdot)$	Tree structure mapping input x to leaf j
w_j^*	Optimal weight assigned to leaf j
λ	Regularization parameter
I_j	Index set of observations assigned to leaf j
I_L, I_R	Index sets of left and right child nodes after a split
$I = I_L \cup I_R$	Union of left and right index sets
<i>Gradients and Hessians</i>	
g_i, h_i	First- and second-order derivatives of $\mathcal{L}$ w.r.t. the tree output (conventional GBDT)
$g_i^{(j)}, h_i^{(j)}$	First- and second-order derivatives of $\mathcal{L}$ w.r.t. the j -th Hyper-Tree parameter
$\bar{G}_j, \bar{H}_j$	Aggregated gradients and Hessians across observations in leaf j
K	Number of Gaussian probes in the Hutchinson diagonal Hessian estimator

continued on next page

Table A1 – continued from previous page

Symbol	Description
<i>Target model parameters</i>	
Unhatted parameter symbols denote the true time-varying parameters of the target model, modeled as functions of features. Hats denote Hyper-Tree estimates, or estimates used in forecasts.	
$\theta_{\mathcal{T}}, \hat{\theta}_{\mathcal{T}}$	Target model parameters of the Hyper-Tree and their estimates
$\theta_{\mathcal{N}}, \hat{\theta}_{\mathcal{N}}$	Target model parameters of the Hyper-TreeNet (via the MLP) and their estimates
$\hat{\theta}_{i\mathcal{T}}^{(j)}$	Estimated j -th target model parameter for observation i
$\theta_{j,it}(\mathbf{x}_{it})$	AR(p) coefficient for lag j , series i , time t (panel / global), $j = 1, \dots, p$
$\theta_{j,t}(\mathbf{x}_t)$	AR(p) coefficient for lag j at time t (local / single-series), $j = 1, \dots, p$
$\theta_{j,t}^{\mu}(\mathbf{x}_t)$	Parameter of the mean component (distributional extension)
$\theta_{j,t}^{\sigma}(\mathbf{x}_t)$	Parameter of the standard deviation component (distributional extension)
$\mu_t, \hat{\mu}_t$	Mean of the target distribution and its estimate
$\sigma_t, \hat{\sigma}_t$	Standard deviation of the target distribution and its estimate
$\hat{y}_{t+h t}$	h -step-ahead point forecast conditional on information at time t
<i>STL-specific parameters</i>	
$a_{0,t}(\mathbf{x}_t)$	Intercept of the trend component (STL example)
$a_{1,t}(\mathbf{x}_t)$	Slope of the trend component (STL example)
$c_{r,t}(\mathbf{x}_t)$	Sine weight of the r -th seasonal component (STL example)
$d_{r,t}(\mathbf{x}_t)$	Cosine weight of the r -th seasonal component (STL example)
r	Seasonal component index, $r = 1, \dots, N_{\text{season}}$
N_{season}	Number of seasonal components in the Fourier decomposition
<i>ETS-specific parameters</i>	
$\ell_t, \hat{\ell}_t$	Level component and its estimate
$b_t, \hat{b}_t$	Trend component and its estimate
$s_t, \hat{s}_t$	Seasonality component and its estimate
$\alpha_t(\mathbf{x}_t)$	Level smoothing parameter
$\beta_t(\mathbf{x}_t)$	Trend smoothing parameter
$\gamma_t(\mathbf{x}_t)$	Seasonality smoothing parameter
$\phi_t(\mathbf{x}_t), \hat{\phi}_t(\mathbf{x}_t)$	Trend dampening parameter and its estimate
m_{season}	Frequency of the seasonal cycle in the STL & ETS specifications
k_{season}	Seasonality repeat index in the h -step-ahead ETS forecast
u	Summation index in the h -step-ahead ETS forecast, $u = 1, \dots, h$
v	Product index for cumulative dampening, $v = 1, \dots, u$
<i>Hyper-TreeNet components</i>	
$\mathcal{E}_{\mathcal{T}}, \hat{\mathcal{E}}_{\mathcal{T}}$	Tree embeddings generated by the GBDT component and its estimate
$\mathbf{W}_{\text{proj}}$	Random projection matrix, $\mathbf{W}_{\text{proj}} \in \mathbb{R}^{k \times d}$
$\mathbf{z}$	Expanded representation passed to the MLP, $\mathbf{z} \in \mathbb{R}^{k \times N}$
d	Dimension of tree embeddings
k	Output dimension of the projection (equal to number of target model parameters)

Appendix B. Datasets

Table B1: Dataset Descriptions.

Dataset	# Series	Frequency	Series Length			Lags	Forecast Horizon
			min	avg	max		
Air Passengers	1	monthly	144	144	144	12	12
Australian Electricity Demand	5	monthly	158	159	160	12	24
Australian Retail Turnover	133	monthly	441	441	441	12	24
M3 Monthly	1,428	monthly	66	117	144	12	18
M3 Yearly	645	yearly	20	28	47	3	6
M5	70	daily	1,968	1,969	1,969	14	28
Rossmann Store Sales	1,115	daily	592	757	942	21	40
Tourism	366	monthly	91	299	333	12	24

Appendix C. Hyper-Parameters

Hyper-Tree-AR, Hyper-TreeNet-AR, and Hyper-Tree-ETS additionally use `deterministic=True` and `force_row_wise=True` to ensure reproducibility across runs, though these settings may increase training time.¹¹ For the Hyper-Tree-ETS model, the Hutchinson diagonal estimator uses $K = 5$ Gaussian probes across all experiments.

Table C1: Local Model Hyper-Parameters.

Air Passengers				
Model	Learning Rate	Linear Tree	Epochs	Loss
Hyper-Tree-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
Hyper-Tree-ETS	$1e-03$	<i>False</i>	100	$\mathcal{L}_2$
Hyper-TreeNet-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM-STL	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
Australian Electricity Demand				
Model	Learning Rate	Linear Tree	Epochs	Loss
Hyper-Tree-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
Hyper-Tree-ETS	$1e-01$	<i>False</i>	100	$\mathcal{L}_2$
Hyper-TreeNet-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM-STL	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
Australian Retail Turnover				
Model	Learning Rate	Linear Tree	Epochs	Loss
Hyper-Tree-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
Hyper-Tree-ETS	—	—	—	—
Hyper-TreeNet-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM-STL	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
Tourism				
Model	Learning Rate	Linear Tree	Epochs	Loss
Hyper-Tree-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
Hyper-Tree-ETS	—	—	—	—
Hyper-TreeNet-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM-AR	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$
LightGBM-STL	$1e-01$	<i>True</i>	100	$\mathcal{L}_2$

Unlisted hyper-parameters use default values. $\mathcal{L}_2$ denotes MSE-Loss. Entries with '—' indicate that the model was not trained.

¹¹See <https://lightgbm.readthedocs.io/en/latest/Parameters.html> for details.

Table C2: Global Model Hyper-Parameters.

Australian Electricity Demand									
Model	Learning Rate	Linear Tree	Epochs	Hidden Layers	Batch Size	# Layers	# Heads	Context Length	Loss
Deep-AR	1e-03	—	100	128	128	3	—	2h	NLL_T
Hyper-Tree-AR	1e-01	True	100	—	—	—	—	—	$\mathcal{L}_2$
Hyper-Tree-ETS	1e-02	False	100	—	—	—	—	—	$\mathcal{L}_2$
Hyper-TreeNet-AR	1e-01 / 1e-03	True	100	128	—	—	—	—	$\mathcal{L}_2$
LightGBM	1e-01	True	100	—	—	—	—	—	$\mathcal{L}_2$
LightGBM-AR	1e-01	True	100	—	—	—	—	—	$\mathcal{L}_2$
TFT	1e-03	—	100	128	128	—	4	2h	$\tau_{0.5}$
Australian Retail Turnover									
Model	Learning Rate	Linear Tree	Epochs	Hidden Layers	Batch Size	# Layers	# Heads	Context Length	Loss
Deep-AR	1e-03	—	500	128	128	3	—	2h	NLL_T
Hyper-Tree-AR	1e-01	True	500	—	—	—	—	—	$\mathcal{L}_2$
Hyper-Tree-ETS	1e-01	True	100	—	—	—	—	—	$\mathcal{L}_2$
Hyper-TreeNet-AR	1e-01 / 1e-03	True	500	128	—	—	—	—	$\mathcal{L}_2$
LightGBM	1e-01	True	200	—	—	—	—	—	$\mathcal{L}_2$
LightGBM-AR	1e-01	True	200	—	—	—	—	—	$\mathcal{L}_2$
TFT	1e-03	—	500	128	128	—	4	2h	$\tau_{0.5}$
M3 Monthly									
Model	Learning Rate	Linear Tree	Epochs	Hidden Layers	Batch Size	# Layers	# Heads	Context Length	Loss
Deep-AR	1e-03	—	200	128	128	3	—	2h	NLL_T
Hyper-Tree-AR	1e-01	True	200	—	—	—	—	—	$\mathcal{L}_2$
Hyper-Tree-ETS	—	—	—	—	—	—	—	—	—
Hyper-TreeNet-AR	1e-01 / 1e-03	True	200	128	—	—	—	—	$\mathcal{L}_2$
LightGBM	1e-01	False	500	—	—	—	—	—	$\mathcal{L}_2$
LightGBM-AR	1e-01	False	500	—	—	—	—	—	$\mathcal{L}_2$
TFT	1e-03	—	200	128	128	—	4	2h	$\tau_{0.5}$
M3 Yearly									
Model	Learning Rate	Linear Tree	Epochs	Hidden Layers	Batch Size	# Layers	# Heads	Context Length	Loss
Deep-AR	1e-03	—	100	128	128	3	—	2h	NLL_T
Hyper-Tree-AR	1e-02	True	100	—	—	—	—	—	$\mathcal{L}_2$
Hyper-Tree-ETS*	1e-01	False	100	—	—	—	—	—	$\mathcal{L}_2$
Hyper-TreeNet-AR	1e-03 / 1e-03	True	100	128	—	—	—	—	$\mathcal{L}_2$
LightGBM	1e-01	True	100	—	—	—	—	—	$\mathcal{L}_2$
LightGBM-AR	1e-01	True	100	—	—	—	—	—	$\mathcal{L}_2$
TFT	1e-03	—	100	128	128	—	4	2h	$\tau_{0.5}$
M5									
Model	Learning Rate	Linear Tree	Epochs	Hidden Layers	Batch Size	# Layers	# Heads	Context Length	Loss
Deep-AR	1e-03	—	100	128	128	3	—	2h	NLL_T
Hyper-Tree-AR	1e-02	True	100	—	—	—	—	—	$\mathcal{L}_2$
Hyper-Tree-ETS	—	—	—	—	—	—	—	—	—
Hyper-TreeNet-AR	1e-02 / 1e-03	True	100	128	—	—	—	—	$\mathcal{L}_2$
LightGBM	1e-01	True	100	—	—	—	—	—	$\mathcal{L}_2$
LightGBM-AR	1e-01	True	100	—	—	—	—	—	$\mathcal{L}_2$
TFT	1e-03	—	100	128	128	—	4	2h	$\tau_{0.5}$
Rossmann Store Sales									
Model	Learning Rate	Linear Tree	Epochs	Hidden Layers	Batch Size	# Layers	# Heads	Context Length	Loss
Deep-AR	1e-03	—	500	128	128	3	—	2h	NLL_T
Hyper-Tree-AR	1e-02	True	500	—	—	—	—	—	$\mathcal{L}_2$
Hyper-Tree-ETS	—	—	—	—	—	—	—	—	—
Hyper-TreeNet-AR	1e-02 / 1e-03	True	500	128	—	—	—	—	$\mathcal{L}_2$
LightGBM	1e-01	True	500	—	—	—	—	—	$\mathcal{L}_2$
LightGBM-AR	1e-01	True	500	—	—	—	—	—	$\mathcal{L}_2$
TFT	1e-03	—	500	128	128	—	4	2h	$\tau_{0.5}$
Tourism									
Model	Learning Rate	Linear Tree	Epochs	Hidden Layers	Batch Size	# Layers	# Heads	Context Length	Loss
Deep-AR	1e-03	—	500	128	128	3	—	2h	NLL_T
Hyper-Tree-AR	1e-01	True	500	—	—	—	—	—	$\mathcal{L}_2$
Hyper-Tree-ETS	2e-02	True	100	—	—	—	—	—	$\mathcal{L}_2$
Hyper-TreeNet-AR	1e-01 / 1e-03	True	500	128	—	—	—	—	$\mathcal{L}_2$
LightGBM	1e-01	True	500	—	—	—	—	—	$\mathcal{L}_2$
LightGBM-AR	1e-01	True	500	—	—	—	—	—	$\mathcal{L}_2$
TFT	1e-03	—	500	128	128	—	4	2h	$\tau_{0.5}$

H : Forecast Horizon, $\mathcal{L}_2$: MSE-Loss, NLL_T : Negative log-likelihood of Student-T distribution, $\tau_{0.5}$: quantile loss evaluated at median. For Hyper-TreeNet-AR, the learning rate shows tree-part / MLP-part. Unlisted hyper-parameters use default values. Entries with '—' indicate that the model was not trained. For the M3 Yearly dataset, the Hyper-Tree-ETS parameterizes an ETS model with a linear trend only, as indicated by '*'. For all other applicable datasets, the Hyper-Tree-ETS generates parameters for ETS models with multiplicative seasonality and a damped trend.

Appendix D. Example Forecasts

Figure D1: Global Model Forecasts.

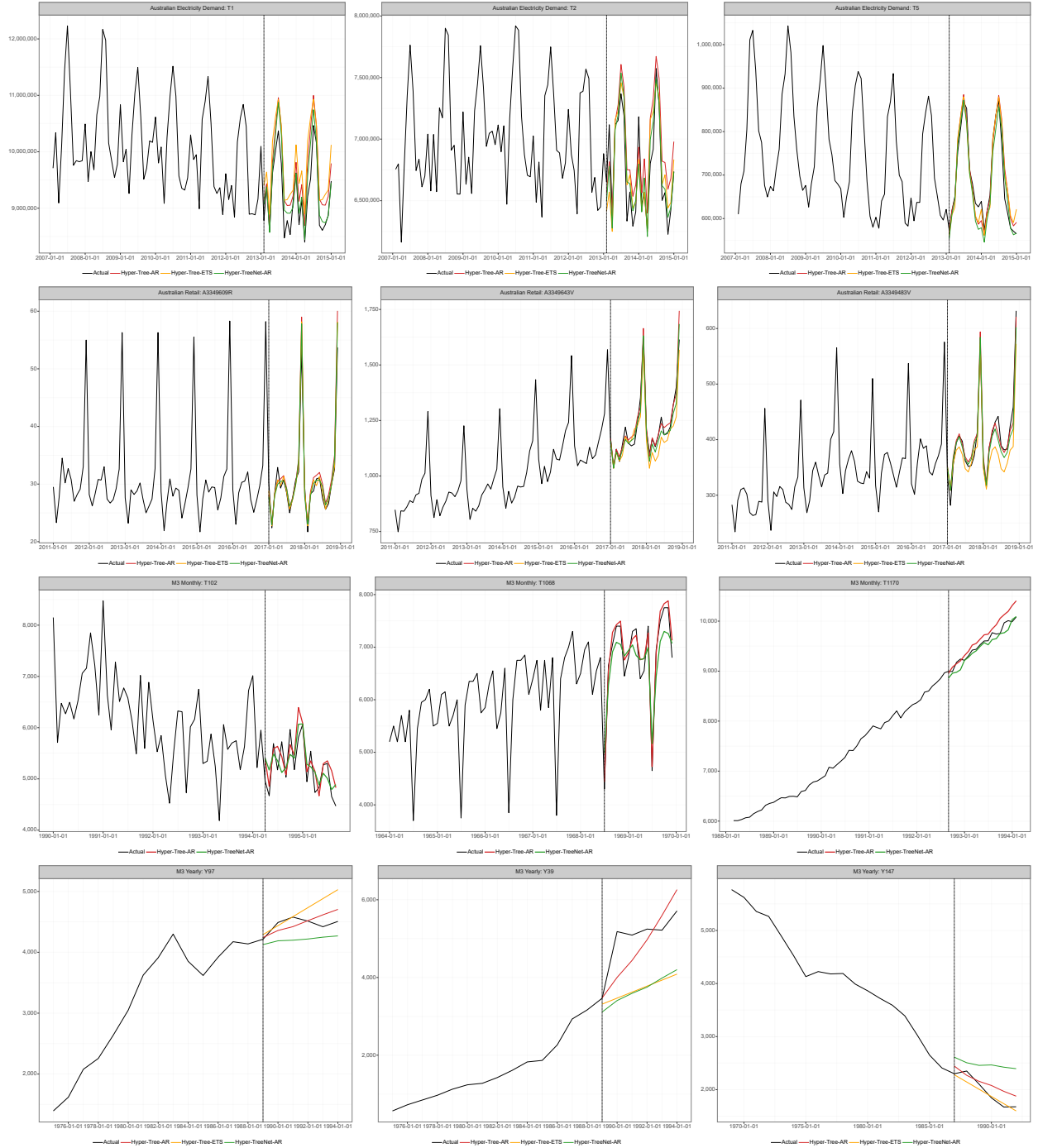
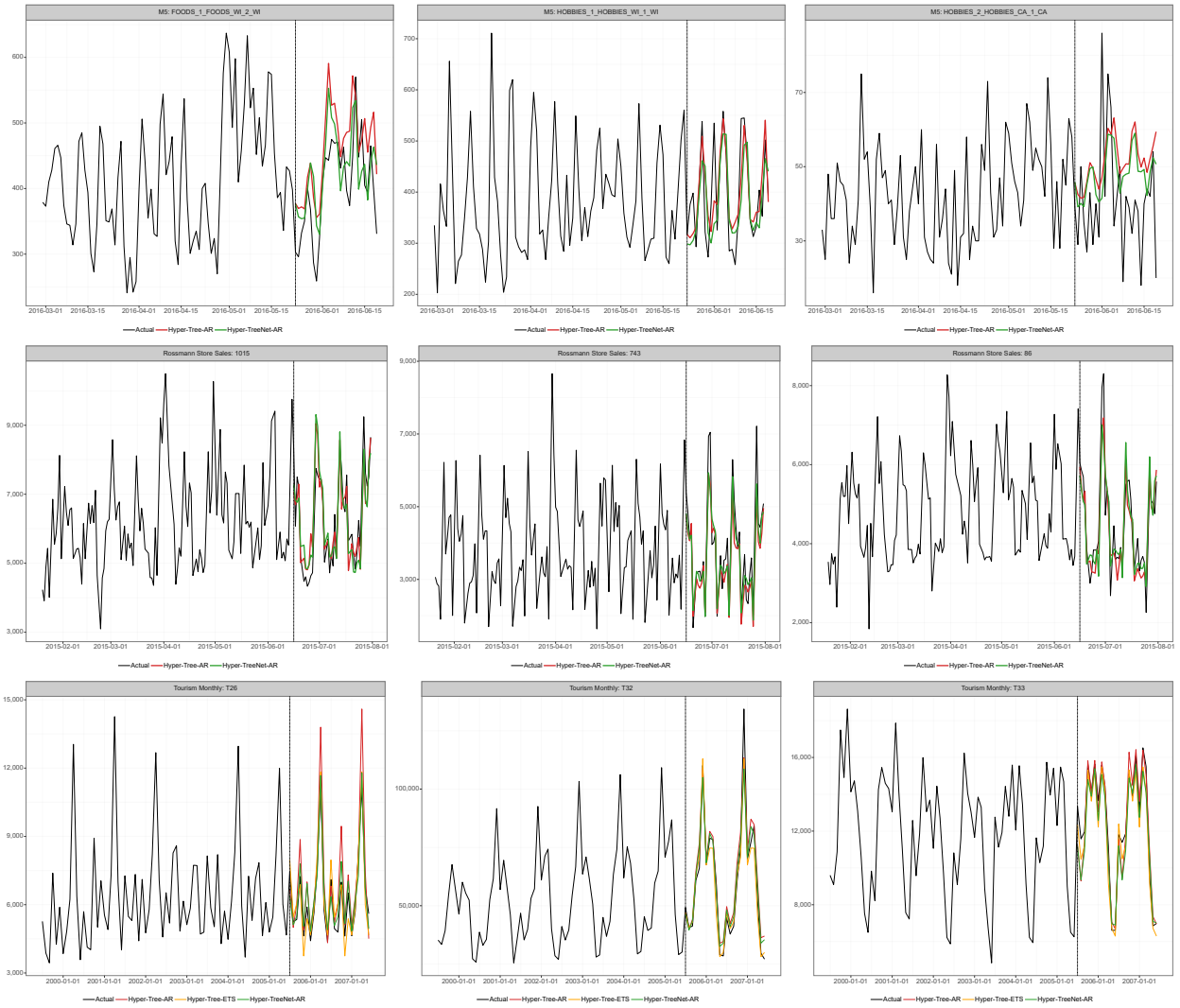


Figure D1 (continued).



Appendix E. Gradient Flow Strategies in Hybrid Tree-Network Models

When integrating GBDTs with neural networks in our Hyper-TreeNet architecture, it is important to consider how to reconcile their different learning paradigms. GBDTs operate in a discrete optimization space with threshold-based splits creating piece-wise constant regions, while neural networks optimize in a continuous parameter space with smooth gradient fields. Combining them into a joint framework requires careful consideration of how gradient information flows between the model components during training. In the following, we outline and discuss two possible approaches, each with different implications for the learning characteristics of the individual components: a fully shared gradient flow between the network and GBDT (*Option 1*) and parameter updates within the same iteration but with separated gradient flows (*Option 2*). Both options are sketched in Table E1 with pseudo-codes and exemplified for a Hyper-TreeNet-AR(p) model.

Table E1: Pseudocode of two Estimation Strategies for Hyper-TreeNet-AR(p): Shared vs. Separate Gradient Flows.

Option 1 Shared Gradient	Option 2 Separate Gradient
Require: features, target, lags, gbdt, network for iter = 1 to num_iter do : GBDT Embeddings gbdt_embeds = gbdt(features) Update Network optimizer.zero_grad() ar_params = network(gbdt_embeds) forecasts = forecast(ar_params, lags) loss = loss_fn(forecasts, target) loss.backward() Update GBDT grad, hess = derivatives(loss, gbdt_embeds) optimizer.step() end for	Require: features, target, lags, gbdt, network for iter = 1 to num_iter do : GBDT Embeddings gbdt_embeds = gbdt(features) Update Network network.train() optimizer.zero_grad() ar_params = network(gbdt_embeds) forecasts_net = forecast(ar_params, lags) loss_net = loss_fn(forecasts_net, target) loss_net.backward() optimizer.step() Update GBDT network.eval() ▷ Inference mode ar_params = network(gbdt_embeds) forecasts_gbdt = forecast(ar_params, lags) loss_gbdt = loss_fn(forecasts_gbdt, target) grad, hess = derivatives(loss_gbdt, gbdt_embeds) end for

For *Option 1*, the gradients flow through the GBDT and network in a single backpropagation pass. This provides a fully integrated optimization where both the tree and the network are updated jointly. While this approach offers conceptual elegance through its complete end-to-end training, it may not preserve the distinctive characteristics of each model component. Forcing these different learning paradigms into a single gradient flow could compromise the tree’s discrete decision boundaries and the network’s continuous optimization landscape, potentially diminishing their respective strengths. For *Option 2*, each iteration alternates between two optimization phases: the network parameters are updated first using the current GBDT embeddings, followed by calculating gradients and Hessians for the GBDT using the updated network parameters. This separation of gradient flows reduces the risk of neural network gradients interfering with GBDT updates, allowing each model to maintain its distinctive strengths. While *Option 2* deviates from a full joint optimization, it preserves the core concept of integrated training. The separation potentially enables each component to better leverage its strengths while still benefiting from iterative feedback

between them. The models still inform each other during each iteration, with the GBDT generating informative parameter-space representations from the input features that the network maps to target model parameters. The mutual learning process continues throughout training as GBDT embeddings influence the network’s training and the network’s state guides GBDT updates in each iteration. While both gradient flow options are valid approaches to joint optimization, we default to *Option 2* for all experiments, allowing each model component, GBDT and neural network, to maintain its distinct characteristics while still benefiting from the iterative feedback between them, though users can also select the fully integrated gradient approach in the model implementation.

Appendix F. Time-Varying Parameters

A defining aspect of our Hyper-Tree framework is its ability to estimate time-varying parameters. Unlike conventional time series models that estimate a set of constant, non-varying coefficients, Hyper-Trees dynamically adapt parameters as a function of features, enabling them to respond to evolving environments. This is illustrated in the following figures with visualizations of estimated and forecasted AR-coefficients for the **Air Passengers**, **Rossmann Store Sales** and **M5** datasets. For clarity, we display only a subset of each series’ history, rather than the full series length.

The pronounced seasonal patterns of AR-parameters in Figure F1 show how the models adapt to cyclical behaviors in the underlying series. For the **Rossmann Store Sales** and **M5** datasets, the influence of features is visible in the parameter plots, where distinct shifts and variations correspond to changes in feature values. These parameters continue to vary for the forecast horizon (indicated by dotted vertical lines), demonstrating how our approach maintains adaptivity beyond the training data. This dynamic parameterization allows Hyper-Trees to effectively respond to changing environments without requiring model re-training, providing a distinctive capability over conventional approaches with fixed parameter estimates (Lee et al., 2023).

Figure F1: Time-Varying AR(12) Parameters for **Air Passengers** Dataset.

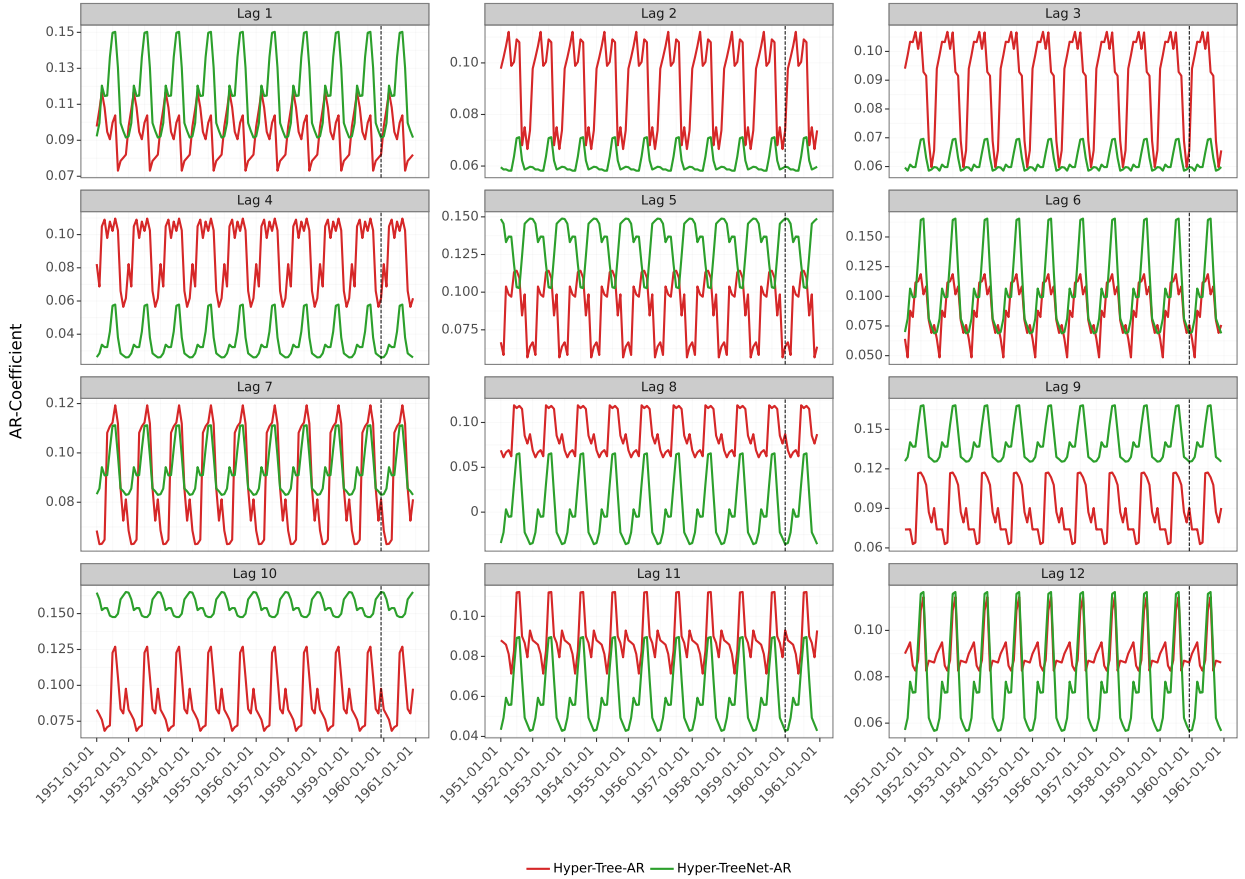


Figure F2: Time-Varying AR(21) Parameters for Rossmann Store Sales Dataset.

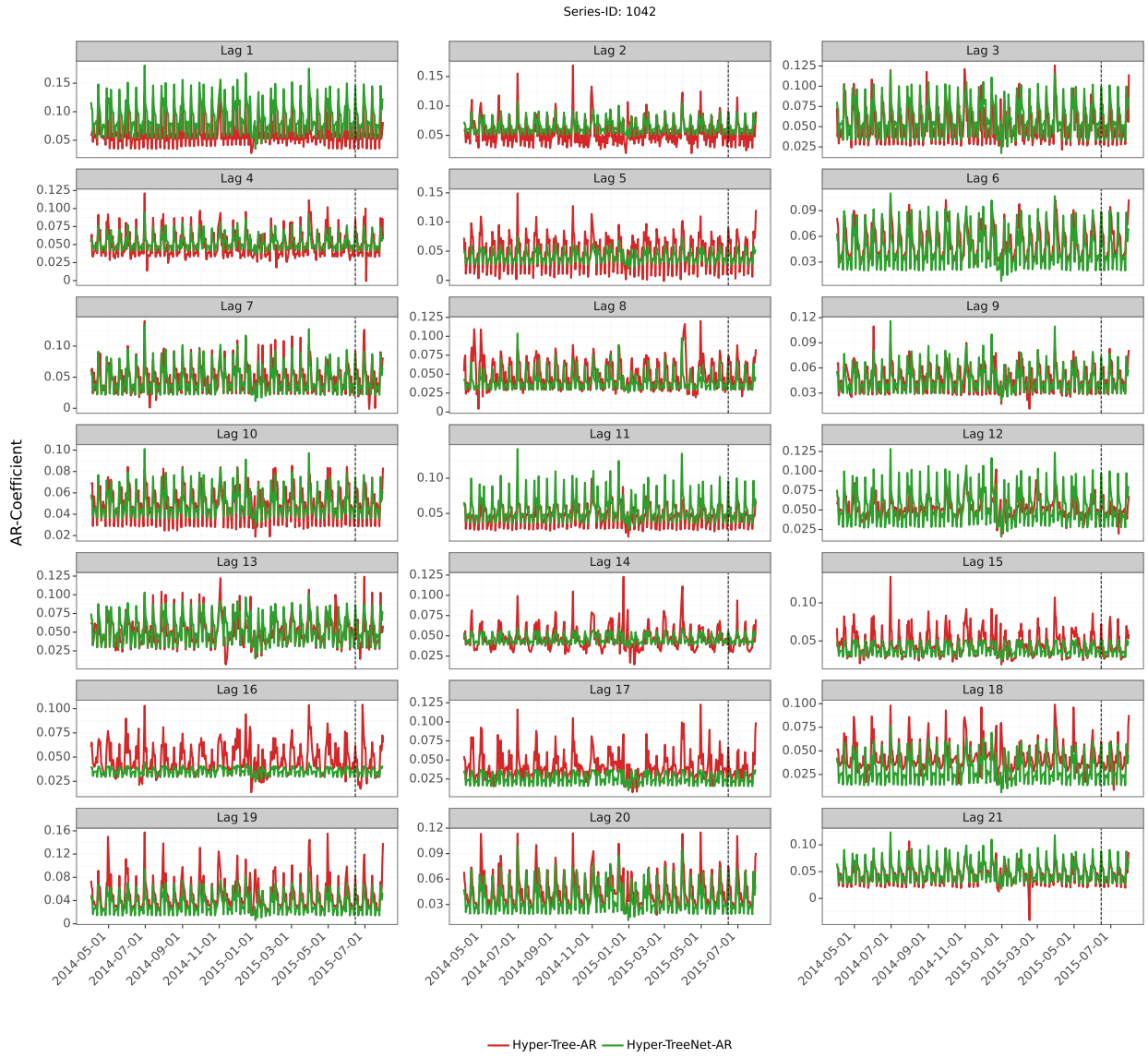


Figure F3: Time-Varying AR(14) Parameters for M5 Dataset.



It is important to note that the parameters estimated by **Hyper-Tree-AR**(p) and **Hyper-TreeNet-AR**(p) models will generally differ from those estimated by conventional $AR(p)$ models. While conventional AR models estimate constant coefficients, Hyper-Trees estimate time-varying parameters that adapt to changing conditions based on features. This difference implies that autoregressive parameters in our framework represent conditional, context-specific relationships rather than fixed global values. Additionally, these dynamic parameters compensate for components not explicitly modeled in the target AR-structure. When the underlying data contains covariate effects, such as sales campaigns in the M5 and **Rossmann** datasets, Hyper-Tree parameters naturally adapt to capture these influences rather than reflecting only the pure autoregressive structure. Another important consideration relates to stationarity conditions. Conventional $AR(p)$ models require parameter values within specific bounds to ensure stationarity, with these conditions assessed globally across the entire series. In our Hyper-Tree framework, however, AR-parameters vary with time and features, creating locally time-varying stationarity properties that may differ across time periods and feature contexts (Dahlhaus, 2000, 2012). This introduces the possibility of non-stationary episodes within an otherwise well-behaved process, requiring careful monitoring of parameter trajectories over time.

Appendix G. Embedding Analysis

The dimensionality of tree embeddings is an important architectural choice in the **Hyper-TreeNet** framework. While we use $d = 1$ as the default for all experiments, this section investigates the role of embedding dimensions in more detail. We first evaluate how the choice of the embedding dimension affects forecast accuracy across multiple datasets, and then also visualize the learned embeddings to understand what patterns they capture.

Appendix G.1. Effect on Forecast Accuracy

To quantify the accuracy-efficiency trade-off, we evaluate embedding dimensions $d \in \{1, 3, 5, 10\}$ in a global time series model setting, varying only d while keeping all remaining hyper-parameters fixed at their global model configurations as reported in Table C2. Table G1 presents results across datasets.

Table G1: Embedding Dimension Analysis for **Hyper-TreeNet-AR**.

Dataset	Embedding-Dim	MAPE	sMAPE	WAPE	RMSE	MAE	MASE	Runtime (Minutes)
Air Passengers	1	4.119	4.144	3.972	21.289	18.915	0.849	0.019
	3	2.474	2.454	2.364	14.737	11.256	0.505	0.023
	5	2.449	2.405	2.342	15.595	11.154	0.501	0.026
	10	2.597	2.532	2.390	16.663	11.382	0.511	0.033
Australian Electricity Demand	1	2.964	2.950	2.960	163,026.9	121,972.3	0.886	0.022
	3	3.255	3.248	3.232	169,567.5	129,566.5	0.959	0.031
	5	3.507	3.456	3.474	189,624.7	158,295.1	1.090	0.039
	10	4.792	4.699	4.693	240,631.8	216,584.5	1.497	0.055
Australian Retail Turnover	1	6.403	6.292	6.427	16.179	13.409	1.090	0.109
	3	9.429	10.135	9.819	34.982	29.894	2.006	0.194
	5	11.208	10.627	11.204	47.466	41.005	2.468	0.261
	10	10.008	11.035	10.192	20.635	16.789	1.730	0.468
M3 Monthly	1	19.778	14.659	14.603	814.225	677.430	1.374	0.075
	3	20.119	14.590	14.568	809.894	673.574	1.357	0.132
	5	19.698	14.235	14.342	793.316	658.416	1.287	0.179
	10	19.672	14.233	14.321	796.733	661.793	1.437	0.364
M3 Yearly	1	21.782	18.389	18.133	1,233.0	1,088.0	1.558	0.031
	3	22.919	18.184	19.000	1,425.5	1,242.2	1.539	0.039
	5	24.506	18.594	20.560	1,697.2	1,454.1	1.589	0.048
	10	24.733	18.672	20.645	1,687.8	1,451.9	1.662	0.075
M5	1	15.290	13.671	13.926	79.713	63.052	0.990	0.049
	3	16.055	13.990	14.378	81.141	65.103	1.017	0.078
	5	16.699	14.457	14.946	86.269	70.111	1.079	0.109
	10	16.737	14.434	14.893	89.909	72.489	1.087	0.199
Rossmann Store Sales	1	9.437	9.597	9.449	857.004	664.496	0.385	0.693
	3	9.616	9.804	9.657	874.438	677.550	0.389	1.435
	5	9.071	9.163	9.099	827.530	637.683	0.367	2.078
	10	9.903	10.219	10.116	913.647	708.252	0.410	3.858
Tourism	1	23.731	20.603	19.587	3,031.1	2,515.8	1.113	0.135
	3	31.894	24.327	24.489	5,120.5	4,346.6	1.554	0.280
	5	40.221	29.981	31.602	6,829.6	5,932.0	2.179	0.376
	10	34.926	29.595	28.542	3,813.3	3,108.7	1.743	0.694

Reported are the mean forecast errors across series per dataset and total runtimes in minutes, with lower values indicating better performance and the best metrics highlighted in bold. Mean Absolute Percentage Error (MAPE); Symmetric Mean Absolute Percentage Error (sMAPE); Weighted Absolute Percentage Error (WAPE); Root Mean Squared Error (RMSE); Mean Absolute Error (MAE); Mean Absolute Scaled Error (MASE), which is calculated relative to the **AutoETS** forecasts (MAE of model / MAE of **AutoETS**). All experiments use identical hyper-parameters as reported in Table C2. The **Air Passengers** dataset contains a single series, for which the local and global model settings are equivalent.

Despite its simplicity, $d = 1$ achieves the best or near-best accuracy on most datasets. The **Air Passengers** dataset shows the most pronounced benefit from higher embedding dimensions, while **Rossmann** and **M3 Monthly** show smaller gains. Several factors might explain this pattern.

First, the GBDT’s recursive partitioning captures feature interactions implicitly through root-to-leaf decision paths, with successive splits across categorical, numeric, and temporal features. Hence, each leaf corresponds to a region of the feature space (e.g., ‘weekday, promotion, store type A’), and the embeddings capture this compressed context, representing complex non-linear relationships and interaction effects rather than encoding raw features directly. This may explain why a single embedding dimension can often be sufficient, since the trees already provide an interaction-rich feature representation. The random projection layer in our **Hyper-TreeNet** architecture provides another key mechanism that compensates for low-dimensional tree embeddings. As described in Section 2.2, this layer expands the d -dimensional tree embeddings to k dimensions (where k equals the number of target model parameters) before the MLP processes it, effectively creating k views of the original embeddings. This diversity proves essential: without the expansion, forecast accuracy deteriorates, as ablation study **A4** in Table 4 demonstrates. The random projection hence serves as an effective bridge between the compact tree embeddings and the MLP, providing sufficient representational diversity even when $d = 1$. Beyond the role of random projections, higher embedding dimensions increase the overall model capacity and introduce additional learning targets in the GBDT component. For datasets with limited complexity or sample size, this additional capacity may not be warranted and can lead to degraded generalization. In such cases, the constraint imposed by $d = 1$ acts as a beneficial information bottleneck, forcing the tree ensemble to compress information into a single dimension that captures the most salient patterns while discarding noise.

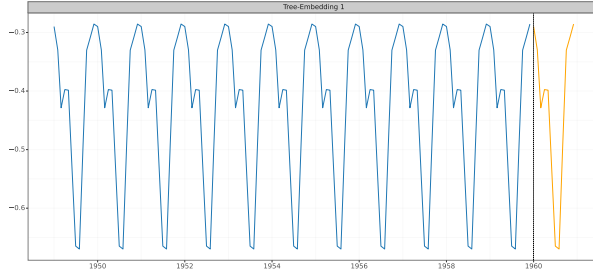
However, for datasets with rich feature spaces, this same compression can become a limiting factor. The **Rossmann** dataset illustrates this case: it contains a rich set of categorical features (store types, assortment levels, promotional indicators, ...) and temporal features (day-of-week, holiday indicators, school holiday patterns, ...). When the feature space is sufficiently complex and the dataset large enough to support learning additional parameters, $d = 1$ may impose an overly restrictive bottleneck, forcing the GBDT to compress complex feature information into a single embedding. The accuracy gains with higher-dimensional embeddings can be understood through the lens of representational capacity: with $d > 1$, the tree ensemble can encode multiple complementary aspects of the feature space, enabling richer downstream parameter adaptation. Interestingly, the **Air Passengers** dataset also benefits from higher embedding dimensions (WAPE improving from 3.972 at $d = 1$ to 2.342 at $d = 5$), despite being a single series with only temporal features. Here, the benefit appears to stem not from feature complexity but rather from the dataset’s pronounced seasonality: with $d > 1$, the model can encode complementary aspects of the seasonal pattern, rather than compressing all temporal information into a single dimension. For higher dimensions ($d > 5$), a different pattern emerges. Across the datasets evaluated, except for modest gains on the **M3 Monthly** dataset, $d = 10$ does not yield accuracy improvements over lower dimensions. This suggests diminishing returns: beyond a certain point, additional embedding capacity no longer captures meaningful structure but instead seems to increase the risk of overfitting or training instability. The same pattern is also apparent on **Australian Retail Turnover** and **Tourism**, where $d = 5$ reduces accuracy substantially.

Based on these results, we offer the following general guidance for practitioners. For most forecasting applications with modest feature sets and moderate dataset sizes, $d = 1$ provides an effective default choice, offering computational efficiency while achieving competitive accuracy. The random projection mechanism ensures adequate representational capacity despite the low embedding dimensionality. Intermediate embedding dimensions ($d = 3$) typically perform within a few percentage points of the best variant on most datasets, providing a reasonable middle ground between the efficiency of $d = 1$ and the additional capacity of $d = 5$. For complex operational datasets with rich categorical features, $d = 5$ may yield meaningful accuracy improvements, though as the **Australian Retail Turnover** and **Tourism** results show, this is not guaranteed and validation against lower embedding dimensions is advisable. Beyond $d = 5$, dimensions generally show diminishing returns and sometimes degrade performance further, with $d = 10$ rarely improving over lower dimensions. Given the dataset-dependent nature of optimal embedding dimensionality and the absence of reliable a priori guidance, practitioners without strong assumptions on their dataset characteristics may prefer to treat d as a hyper-parameter, with $d \in \{1, 3, 5\}$ providing a reasonable search range.

Appendix G.2. Visualizing Tree Embeddings

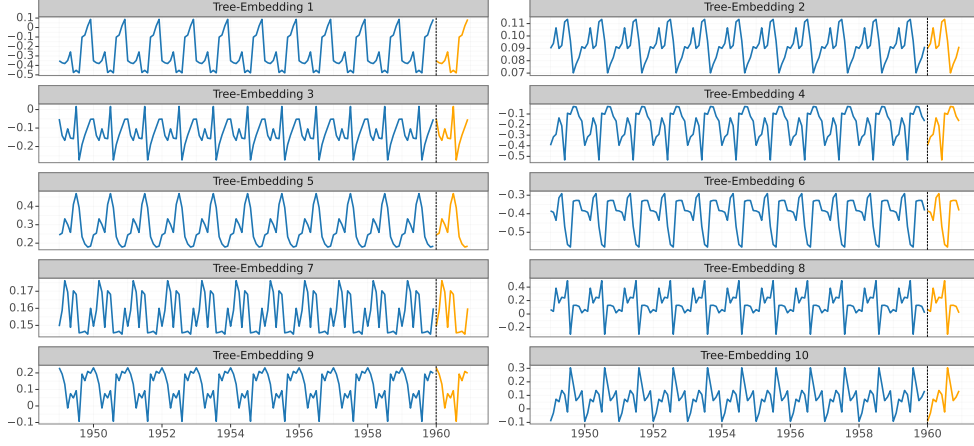
The quantitative results in the preceding section evaluate how the choice of the embedding dimension affects forecast accuracy. To provide insight into the internal representations learned by the **Hyper-TreeNet-AR** model, we now visualize the tree embeddings for the **Air Passengers** dataset using both $d = 1$ and $d = 10$ configurations. We selected this dataset for its pronounced and regular seasonality, which facilitates interpretation of how embeddings encode temporal patterns.

Figure G1: Single tree embedding ($d = 1$) for the **Air Passengers** dataset. Blue indicates training period, orange indicates forecast period.



The single tree embedding in Figure G1 shows a regular 12-month oscillation. Since the only features are time-derived temporal indicators (month, quarter), identical months across different years map to identical embedding values, producing the observed year-over-year consistency. This visually confirms the discussion in Section 5.5 that similar feature contexts yield similar embeddings, preventing arbitrary parameter variation. The transition from training (blue) to forecast (orange) period shows no discontinuity, with the embedding continuing its seasonal oscillation naturally because the temporal features follow the same 12-month cycle. The single embedding compresses all seasonal information into one dimension, forcing the model to represent the annual cycle along a single axis. Despite this compression, the model achieves reasonable accuracy, demonstrating that the random projection layer can partially compensate for low-dimensional embeddings by expanding this single signal into multiple views for the MLP.

Figure G2: Multiple tree embeddings ($d = 10$) for the **Air Passengers** dataset. Blue indicates training period, orange indicates forecast period.



With $d = 10$, a more diverse set of patterns emerges. Most embeddings exhibit clear seasonal behavior, but with distinct characteristics: some show opposite phases, with one peaking where another troughs, while others differ in amplitude or shape. This diversity allows the model to represent complementary aspects of the seasonal structure that a single embedding would compress into one dimension. However, not every embedding is equally amenable to interpretation. For example, embeddings 7 and 9 show oscillatory behavior, but the signal is more subtle and does not align as directly with a single seasonal component. Comparing the two configurations hence reveals a trade-off between interpretability and accuracy. The single embedding ($d = 1$) offers a clear, immediately interpretable representation of the seasonal cycle, allowing practitioners to directly read off seasonal effects from the embedding. The $d = 10$ configuration sacrifices this clarity for improved accuracy, distributing seasonal information across multiple dimensions that are individually harder to interpret but collectively more expressive.¹² Compared to the single embedding, the higher-dimensional representation achieves higher accuracy (WAPE: 2.390 vs. 3.972 for $d = 1$). However, consistent with Table G1, $d = 5$ achieves the highest accuracy (WAPE: 2.342), with $d = 10$ showing slight degradation. This aligns with our general finding that moderate to low embedding dimensions often provide the best balance between representational capacity and generalization. The relatively small gap in accuracy between $d = 5$ and $d = 10$ on this dataset likely reflects the pronounced seasonality of the **Air Passengers** dataset, where additional embedding capacity captures genuine seasonal variation rather than noise. As with $d = 1$, all embeddings show smooth continuation from training to forecast periods, confirming that the feature-driven parameterization generalizes well to future time points with similar feature contexts.

¹²Users of the **Hyper-TreeNet-AR** model have access to SHAP values (Lundberg & Lee, 2017; Lundberg et al., 2020), which quantify how individual features influence each embedding dimension.

Appendix H. Hardware Specifications

All experiments were conducted on a local machine with the following specifications:

- CPU: 13th Gen Intel(R) Core(TM) i9-13900H
- CPU: Cores: 14
- RAM: 64 GB
- GPU: NVIDIA RTX 3500 Ada Generation Laptop GPU
- GPU Memory: 12 GB